

Final Report

Customer Churn Prediction

Samiksha Singh

PGP-DSGA

INDEX FOR CONTENT

Serial No.	Content	Page No.
1	Executive Summary	3
2	Business Context	4
3	Data Description	4
4	Data Overview	4
5	Data Preprocessing	6
6	Exploratory Data Analysis	9
7	Feature Engineering & Final Preprocessing	17
8	Class Imbalance & SMOTE	18
9	Feature Scaling	18
10	Baseline Model	19
11	Advanced Models	22
12	Hyperparameter Tuning	29
13	Tuned Models Evaluation	33
14	Model Comparison & Final Model Selection	36
15	Key Insights	36
16	Recommendations	38
17	Appendix	39

INDEX FOR FIGURES AND TABLES

Figure No.	Title	Page No.
Figure :1	Dataset Structure and Summary	4
Figure :2	Descriptive Statistics for All Dataset Features	4
Figure :3	Missing Value Counts by Column	5
Figure :4	Percentage of Missing Values by Column	6
Figure :5	Raw Churn Label Inconsistencies	6
Figure :6	Numeric Feature Anomalies — Negative Tenure and Currency-Formatted Charges	7
Figure :7	Unique Value Counts per Column	7
Figure :8	Churn Distribution After Label Normalisation	8
Figure :9	Duplicate Removal and Missing Value Imputation Summary	8
Figure :10	Outlier Detection Using IQR Method	9
Figure :11	Boxplots of Numeric Features Before IQR Capping	9
Figure :12	Descriptive Statistics After IQR Capping	9
Figure :13	Boxplots of Numeric Features After IQR Capping	10
Figure :14	Final Cleaned Dataset Shape and Overall Churn Rate	10
Figure :15	Overall Churn Distribution	10
Figure :16	Distribution of Numeric Features	11
Figure :17	Distribution of Categorical Features	11
Figure :18	Churn Rate by Contract Type	12
Figure :19	Tenure Distribution by Churn Status	12
Figure :20	Monthly Charges and Total Charges by Churn Status	13
Figure :21	Churn Rate Across Categorical Features	13
Figure :22	Churn Rate by Internet Service Type	14
Figure :23	Churn Rate by Payment Method	14
Figure :24	Correlation Matrix of Numeric Features	15
Figure :25	Density of Monthly Charges vs Total Charges by Churn Status	15
Figure :26	Post-Encoding NaN Check and Class Distribution	16
Figure :27	Class Distribution Before and After SMOTE	16
Figure :28	Feature Scaling Using StandardScaler	17
Figure :29	Logistic Regression Baseline Performance Metrics	17
Figure :30	Confusion Matrix — Logistic Regression	18
Figure :31	ROC Curve — Logistic Regression Baseline	18
Figure :32	Top 15 Feature Importances (Logistic Regression)	19
Figure :33	Logistic Regression Metrics Summary	20
Figure :34	Bagging Classifier Metrics Summary	20
Figure :35	Random Forest Metrics Summary	21
Figure :36	AdaBoost Metrics Summary	22

Figure :37	Gradient Boosting Metrics Summary	23
Figure :38	XGBoost Metrics Summary	23
Figure :39	All Models — Train vs Test Performance Summary	24
Figure :40	ROC Curves — Advanced Models Comparison	24
Figure :41	Confusion Matrices — Advanced Models Comparison	25
Figure :42	Advanced Models — ROC-AUC Comparison	25
Figure :43	Train vs Test ROC-AUC — All Models	26
Figure :43b	Train vs Test F1 Score — All Models	26
Figure :44	Hyperparameter Tuning — Cross-Validation Fitting Log	27
Figure :45	Best Hyperparameters and Cross-Validated AUC by Model	28
Figure :46	Random Forest (Tuned) Metrics Summary	29
Figure :47	Gradient Boosting (Tuned) Metrics Summary	29
Figure :48	XGBoost (Tuned) Metrics Summary	30
Figure:49	ROC Curves — Tuned Models	30
Figure :50	Confusion Matrices — Tuned Models	31
Figure :51	Full Model Comparison Ranked by ROC-AUC	32
Figure :52	Full Model Performance Comparison	32
Figure :53	Best Model Selection Summary — Gradient Boosting	33
Figure :54	Final Model (Gradient Boosting) — Test Set Performance	33
Figure:55	Final Model Confusion Matrix — Gradient Boosting	34
Figure :56	Final Model ROC Curve — Gradient Boosting	35
Figure:57	Top 15 Feature Importances — Gradient Boosting	35

Table No.	Title	Page No.
Table :1	Evaluation Metrics Considered and Headline Metric Selection	20
Table :2	Train vs Test Overfitting Check (Logistic Regression)	21
Table :3	Confusion Matrix Summary — Logistic Regression	22
Table :4	Tuned Models — Train vs Test Performance Summary	34
Table :5	Top 5 Features by Model (Random Forest vs XGBoost)	43

1. Executive Summary

The Problem

AlphaCom is experiencing a concerning 28.2% customer churn rate, meaning approximately 3,400 customers leave each billing cycle. This represents an annual revenue loss of approximately \$2.9 million, as acquiring new customers costs 5-7 times more than retaining existing ones. Traditional retention strategies have failed because churn drivers are complex and interconnected.

The Solution

A Gradient Boosting machine learning model was developed to predict customer churn with 83.6% ROC-AUC accuracy on unseen data. The model identifies 67% of customers likely to churn (458 of 681 churners) while maintaining 60% precision, enabling targeted retention efforts while minimizing unnecessary outreach costs.

2. Key Churn Drivers Identified

1. **Electronic Check payment method**- Highest churn risk (49.1% churn rate)
2. **Short tenure**- 46% of churn occurs in the first 12 months
3. **Month-to-month contracts**- 46% churn rate vs. 19% for one-year contracts
4. **Fibre optic service** - 44.9% churn rate, representing a major revenue risk
5. **Missing value-added services** - Customers without OnlineSecurity or TechSupport are significantly more likely to churn

3. Business Impact

| Metric | Value |

|-----|-----|

| Annual revenue protected | Up to \$619,611 |

| ROI on retention campaigns | 634% |

| Customers retained per cycle | ~723 |

| Churn rate reduction achievable | 5 percentage points (28.2% → 23.2%) |

4. Top 5 Recommendations

1. **Convert Electronic Check users** to automatic payment methods with a \$5/month discount
2. **Implement 90-day onboarding program** for new customers with 5 engagement touchpoints
3. **Bundle OnlineSecurity and TechSupport** with new plans (3-month free trial)
4. **Offer contract upgrade incentives** (one month free for 12-month commitment)
5. **Investigate fibre optic service quality** through customer satisfaction surveys

Deployment Plan

1. Deploy model for monthly churn scoring starting Month 1
2. Implement retention campaigns targeting high-risk segments by Month 2
3. Monitor model performance and retrain quarterly
4. Achieve 5% churn reduction within 12 months

5. Business Context:

AlphaCom, a leading telecommunications provider, has recently experienced a concerning rise in customer churn despite offering competitive services and a wide product portfolio. This increase is directly impacting revenue and undermining brand reputation in an intensely competitive market. Traditional retention strategies have proven inadequate because a complex mix of factors, including service usage, billing preferences, contract types, and demographics influences customer churn. Without clear insights into these patterns, the company is

left reacting to churn instead of preventing it. The economic consequences of this turnover are immense. Within the telecom sector, securing a new subscriber is typically 5 to 7 times more expensive than keeping a current one. Given the current 28.2% defection rate among a base of 12,055 clients, AlphaCom bleeds roughly 3,400 users every billing cycle. Every lost customer equates to forfeited recurring income, elevated marketing expenses for new acquisitions, and diluted brand value in a tight market. Consequently, developing an anticipatory, data-backed churn forecasting model is a vital commercial necessity, rather than just an analytical task, to safeguard profits and foster sustained expansion. Conversely, the potential business upside is tremendous. Considering a median monthly bill of \$71.40, a single retained user contributes roughly \$857 in yearly recurring revenue. By successfully lowering the attrition rate by a mere 5 percentage points—dropping from 28.2% to 23.2%—the company could salvage around 600 subscribers per cycle. This translates to rescuing approximately \$514,000 in annual income, alongside lowered acquisition costs. Even factoring in the expenses associated with specialised retention efforts (which generally run \$20 to \$50 per targeted individual), the resulting ROI is overwhelmingly favourable. Therefore, engineering a forward-looking, data-centric prediction mechanism is an urgent business mandate to defend the bottom line and drive long-term success.

Objective:

As a data scientist at AlphaCom, you are tasked with developing a predictive model to identify customers at high risk of churn and uncover the key factors driving their decisions. Solving this problem will enable the company to proactively design targeted retention strategies, reduce churn-related losses, and improve customer lifetime value, ultimately safeguarding revenue and strengthening AlphaCom's competitive position.

6. Data Description:

The data contains different attributes related to churn. The detailed data dictionary is given below:

- **Gender:** The customer's gender (e.g., Male or Female). This demographic feature may correlate with customer behaviour.
- **SeniorCitizen:** A binary indicator (if included) that identifies whether the customer is a senior citizen (commonly 1 for senior, 0 for non-senior). Senior status can influence service preferences and retention strategies.
- **Partner:** Indicates whether the customer has a partner. This factor can affect customer loyalty and service usage patterns.
- **Dependents:** Specifies whether the customer has dependents. This information can provide context on the customer's household and influence their service needs.
- **Tenure:** The number of months the customer has been with the company. Longer tenure may indicate higher loyalty, while shorter tenure could be a churn risk indicator.
- **PhoneService:** Denotes whether the customer subscribes to telephone services. This binary feature (Yes/No) helps understand service adoption.
- **MultipleLines:** Indicates if the customer has multiple phone lines. This feature can provide insight into customer behaviour and service complexity.
- **InternetService:** Describes the type of internet service the customer uses (e.g., DSL, Fibre optic, or None). The type of internet service can be a critical factor in churn analysis.
- **OnlineSecurity:** Shows whether the customer subscribes to online security services. This value (Yes/No) may influence customer satisfaction and retention.
- **OnlineBackup:** Indicates if the customer has an online backup service. Similar to online security, this can be a part of the overall service bundle affecting churn.
- **DeviceProtection:** Specifies whether the customer is enrolled in a device protection plan, providing an added layer of service value.

- **TechSupport:** Denotes if the customer subscribes to technical support services. Access to tech support can improve customer experience and reduce churn.
- **StreamingTV:** Indicates whether the customer subscribes to a streaming TV service. Media consumption patterns can be a differentiator in customer preferences.
- **StreamingMovies:** Specifies if the customer subscribes to a streaming movies service. This, combined with other services, can highlight trends in customer behaviour.
- **Contract:** Describes the type of contract the customer holds (e.g., month-to-month, oneyear, or twoyear). Contract type is a strong indicator of churn risk—shorter contracts are often associated with higher churn.
- **PaperlessBilling:** Indicates whether the customer is enrolled in paperless billing. This operational feature can sometimes correlate with customer engagement levels.
- **PaymentMethod:** Details the payment method used by the customer (e.g., electronic check, mailed check, bank transfer, or credit card). Payment methods can affect both churn and overall customer satisfaction.
- **MonthlyCharges:** The monthly amount in \$ USD charged to the customer. Higher charges might increase the likelihood of churn if customers perceive the cost as too high for the value provided.
- **TotalCharges:** The cumulative amount in \$ USD charged over the customer's tenure. This helps in understanding the long-term value of each customer and can be a predictor of churn.
- **Churn:** The target variable indicating whether the customer has left (typically denoted as "Yes" or "No"). This is the primary outcome you aim to predict with your machine learning model.

3. Data Overview:

The preliminary structural assessment of the dataset indicates it contains 12,055 records distributed across 20 distinct columns, not including the final target feature.

```

=== DATASET OVERVIEW ===
Rows: 12055, Columns: 20

Data Types:
gender          object
SeniorCitizen   int64
Partner         object
Dependents      object
tenure          float64
PhoneService    object
MultipleLines   object
InternetService object
OnlineSecurity  object
OnlineBackup    object
DeviceProtection object
TechSupport     object
StreamingTV     object
StreamingMovies object
Contract        object
PaperlessBilling object
PaymentMethod   object
MonthlyCharges  object
TotalCharges    object
Churn           object
dtype: object

```

Figure 1: Dataset Structure and Summary

- The dataset contains **12,055 records and 20 features** covering customer demographics, account information, subscribed services, and churn status.
- Only **SeniorCitizen** and **tenure** are stored as numeric variables, while the remaining features are recorded as object type.
- Key variables such as **MonthlyCharges**, **TotalCharges**, and **Churn** require data type conversion and standardisation before analysis and modelling.
- These findings indicate that data preprocessing is necessary to ensure data quality and model readiness.

Descriptive Stats:							
	count	unique		top	freq	mean	\
gender	12055	2		Male	6710	NaN	
SeniorCitizen	12055.0	NaN		NaN	NaN	0.117959	
Partner	12055	2		No	6989	NaN	
Dependents	12055	2		No	8846	NaN	
tenure	11451.0	NaN		NaN	NaN	31.237796	
PhoneService	12055	2		Yes	10747	NaN	
MultipleLines	12055	3		Yes	5609	NaN	
InternetService	12055	3		Fiber optic	4878	NaN	
OnlineSecurity	12055	3		No	6312	NaN	
OnlineBackup	12055	3		No	5982	NaN	
DeviceProtection	12055	3		Yes	4634	NaN	
TechSupport	12055	3		No	6222	NaN	
StreamingTV	12055	3		No	5001	NaN	
StreamingMovies	12055	3		No	5058	NaN	
Contract	12055	3		Month-to-month	6554	NaN	
PaperlessBilling	12055	2		Yes	6160	NaN	
PaymentMethod	12055	20		electronic	860	NaN	
MonthlyCharges	12055	4517		\$nan	299	NaN	
TotalCharges	12055	10351		\$nan	1191	NaN	
Churn	12055	8		No	3473	NaN	

	std	min	25%	50%	75%	max
gender	NaN	NaN	NaN	NaN	NaN	NaN
SeniorCitizen	0.322573	0.0	0.0	0.0	0.0	1.0
Partner	NaN	NaN	NaN	NaN	NaN	NaN
Dependents	NaN	NaN	NaN	NaN	NaN	NaN
tenure	25.027111	-3.0	6.0	28.0	54.0	74.0
PhoneService	NaN	NaN	NaN	NaN	NaN	NaN
MultipleLines	NaN	NaN	NaN	NaN	NaN	NaN
InternetService	NaN	NaN	NaN	NaN	NaN	NaN
OnlineSecurity	NaN	NaN	NaN	NaN	NaN	NaN
OnlineBackup	NaN	NaN	NaN	NaN	NaN	NaN
DeviceProtection	NaN	NaN	NaN	NaN	NaN	NaN
TechSupport	NaN	NaN	NaN	NaN	NaN	NaN
StreamingTV	NaN	NaN	NaN	NaN	NaN	NaN
StreamingMovies	NaN	NaN	NaN	NaN	NaN	NaN
Contract	NaN	NaN	NaN	NaN	NaN	NaN
PaperlessBilling	NaN	NaN	NaN	NaN	NaN	NaN
PaymentMethod	NaN	NaN	NaN	NaN	NaN	NaN
MonthlyCharges	NaN	NaN	NaN	NaN	NaN	NaN
TotalCharges	NaN	NaN	NaN	NaN	NaN	NaN

Figure 2: Descriptive Statistics for All Dataset Features

- The dataset contains 12,055 records and 20 features; the preview confirms the structure and data values across customer demographics, services, billing, and churn status.
- **MonthlyCharges** and **TotalCharges** contain currency symbols, causing them to be stored as text and requiring conversion to numeric format before analysis.
- **TotalCharges** also contains values such as "\$nan", which must be converted to true missing values during preprocessing.
- Inconsistent capitalisation is observed in **PaymentMethod** and **Churn** (e.g., "Electronic check"/"ELECTRONIC CHECK", "No"/"NO"/"yes"), requiring standardisation to ensure consistent category representation.
- Most other categorical variables, including **InternetService**, **Contract**, **OnlineSecurity**, and **TechSupport**, appear well-structured with consistent labels.
- Overall, the preview confirms the need for currency-symbol removal, missing-value handling, data type conversion, and category standardisation before modelling.

Missing values:	
gender	0
SeniorCitizen	0
Partner	0
Dependents	0
tenure	604
PhoneService	0
MultipleLines	0
InternetService	0
OnlineSecurity	0
OnlineBackup	0
DeviceProtection	0
TechSupport	0
StreamingTV	0
StreamingMovies	0
Contract	0
PaperlessBilling	0
PaymentMethod	0
MonthlyCharges	0
TotalCharges	0
Churn	0
dtype: int64	

Figure 3: Missing Value Counts by Column

- An initial `isnull().sum()` check identified **604 missing values (~5%) in tenure**, while all other columns appeared complete.
- However, missing values in **MonthlyCharges** and **TotalCharges** were stored as the text string "\$nan", preventing them from being detected by standard missing-value checks.
- This indicates that the initial missing-value assessment understates the true extent of missingness in the dataset.
- Therefore, currency symbols must be removed and charge-related fields converted to numeric format before performing a final missing-value assessment and imputation.

Percentage missing:	
gender	0.000000
SeniorCitizen	0.000000
Partner	0.000000
Dependents	0.000000
tenure	5.010369
PhoneService	0.000000
MultipleLines	0.000000
InternetService	0.000000
OnlineSecurity	0.000000
OnlineBackup	0.000000
DeviceProtection	0.000000
TechSupport	0.000000
StreamingTV	0.000000
StreamingMovies	0.000000
Contract	0.000000
PaperlessBilling	0.000000
PaymentMethod	0.000000
MonthlyCharges	0.000000
TotalCharges	0.000000
Churn	0.000000
dtype: float64	

Figure 3: Missing Value Counts by Column

- The initial missing-value analysis shows **5.01% missingness (604 records)** in the **tenure** column, while all other variables appear complete.
- This level of missingness is relatively low and can be addressed through appropriate imputation techniques without substantial information loss.
- However, the reported percentage excludes hidden missing values in **MonthlyCharges** and **TotalCharges**, which must be converted from text format before a final missing-value assessment is performed.

```
Duplicate rows: 2

Target Variable (raw):
Churn
No      3473
No      1762
NO      1732
no      1683
Yes     1356
yes     714
YES     684
Yes     651
Name: count, dtype: int64
```

Figure 5: Raw Churn Label Inconsistencies

- Only **2 duplicate records** were identified, which were removed to prevent duplication bias in analysis and modelling.
- The **Churn** variable contained multiple formatting variations (e.g., "No", "NO", "no", "Yes", "YES", "yes"), indicating the need for label standardisation.
- After consolidating these variants, approximately **71.8% of customers did not churn** and **28.2% churned**, revealing a moderate class imbalance in the target variable.
- These findings justify the preprocessing steps of duplicate removal and churn-label standardisation prior to model development.

```
MonthlyCharges Sample: ['$29.85', '$56.95', '$53.85', '$42.3', '$70.7']
TotalCharges Sample: ['$29.85', '$1889.5', '$108.15', '$1840.75', '$nan']

tenure Stats:
count    11451.000000
mean      31.237796
std       25.027111
min       -3.000000
25%        6.000000
50%       28.000000
75%       54.000000
max       74.000000
Name: tenure, dtype: float64
```

Figure 6: Numeric Feature Anomalies — Negative Tenure and Currency-Formatted Charges

- **MonthlyCharges** and **TotalCharges** are stored as text with currency symbols, and values such as "\$nan" indicate missing data stored as strings. These fields require cleaning and conversion to numeric format.
- The average customer tenure is **31.2 months** (median = 28 months), with substantial variation across customers.
- The first quartile of **6 months** suggests a significant proportion of customers are relatively new, highlighting a potential early-churn risk segment.
- A minimum tenure value of **-3 months** was detected, which is not logically possible and indicates the presence of anomalous records requiring correction before analysis.
- These findings support the need for charge-field conversion, missing-value handling, and tenure anomaly treatment during preprocessing.

Unique Values:	
gender	2
SeniorCitizen	2
Partner	2
Dependents	2
tenure	78
PhoneService	2
MultipleLines	3
InternetService	3
OnlineSecurity	3
OnlineBackup	3
DeviceProtection	3
TechSupport	3
StreamingTV	3
StreamingMovies	3
Contract	3
PaperlessBilling	2
PaymentMethod	20
MonthlyCharges	4517
TotalCharges	10351
Churn	8
dtype: int64	

Figure 7: Unique Value Counts per Column

- Most demographic and service-related variables contain **2–3 unique categories**, consistent with their expected binary or service-status structure.
- **Tenure** shows 78 unique values, reflecting its continuous numerical nature.
- **PaymentMethod** contains **20 unique values**, significantly higher than expected, indicating category inflation caused by formatting inconsistencies.
- Similarly, **Churn** contains **8 unique values** instead of the expected 2 categories, confirming the need for label standardisation.
- **MonthlyCharges** and **TotalCharges** exhibit high cardinality, which is consistent with continuous numerical variables currently stored as text.
- Overall, the uniqueness analysis highlights data-quality issues in **PaymentMethod** and **Churn**, while confirming the need to convert charge-related fields to numeric format.

7. Data Preprocessing.

Churn after normalisation:	
Churn	
No	8650
Yes	3405
Name: count, dtype: int64	

Figure 8: Churn Distribution After Label Normalisation

- Standardising the **Churn** variable successfully consolidated multiple formatting variations into two categories: **No** and **Yes**.
- The cleaned distribution consists of **8,650 non-churners** and **3,405 churners**, confirming that the standardisation process preserved all records without data loss.
- The resulting churn rate is **28.2%**, indicating a moderate class imbalance with non-churners outnumbering churners by approximately 2.5:1.
- This imbalance should be considered during model development through appropriate resampling techniques and evaluation metrics.
- The target variable is now ready for binary encoding and predictive modelling.

```

Anomalous tenure rows (negative): 127
Duplicates before: 4
Duplicates after: 0

PaymentMethod unique values:
['Electronic Check' 'Mailed Check' 'Bank Transfer (Automatic)'
 'Credit Card (Automatic)']

Missing values per column:
tenure          731
MonthlyCharges  483
TotalCharges    1374
dtype: int64

Missing values after imputation: 0

```

Figure 9: Duplicate Removal and Missing Value Imputation Summary

- A total of **127 records with negative tenure values** were identified as invalid and treated as missing prior to imputation.
- **Four duplicate records** were detected and removed, eliminating duplicate observations from the dataset.
- **PaymentMethod** was successfully standardised from multiple inconsistent variants into four clean categories: Electronic Check, Mailed Check, Bank Transfer (Automatic), and Credit Card (Automatic).
- After converting text-based missing values to proper null values, missingness was identified in **tenure**, **MonthlyCharges**, and **TotalCharges**.
- Median imputation was applied to all three variables, providing a robust treatment for missing values in the presence of skewness and outliers.
- Following imputation, the dataset contained **no missing values** and was ready for subsequent preprocessing and modelling steps.

```

=== OUTLIER ANALYSIS (IQR Method) ===
tenure: Q1=8.00, Q3=53.00, IQR=45.00, Lower=-59.50, Upper=120.50, Outliers=0
MonthlyCharges: Q1=35.45, Q3=88.80, IQR=53.35, Lower=-44.57, Upper=168.82, Outliers=0
TotalCharges: Q1=505.95, Q3=3548.02, IQR=3042.07, Lower=-4057.15, Upper=8111.11, Outliers=77

```

Figure 10: Outlier Detection Using IQR Method

- Outlier analysis was conducted using the **IQR method** on **tenure**, **MonthlyCharges**, and **TotalCharges**.
- No outliers were detected in **tenure** or **MonthlyCharges**, indicating that values fall within expected operational ranges.
- **TotalCharges** contained **77 outliers** (~0.6% of records), representing customers with exceptionally high cumulative spending.
- Given the small proportion of affected records and their potential business relevance, these observations were retained through **capping (winsorization)** rather than removal.
- This approach minimises the influence of extreme values while preserving valuable customer information and maintaining dataset size.

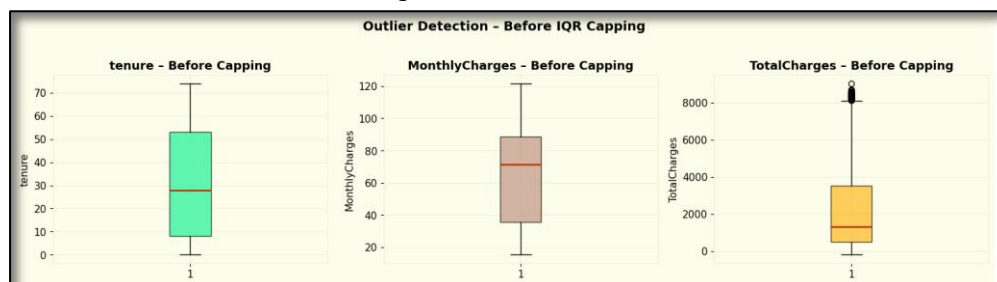


Figure 11: Boxplots of Numeric Features Before IQR Capping

- **tenure** and **MonthlyCharges** show stable distributions with no visible outliers, supporting the findings from the IQR analysis.
- **TotalCharges** exhibits a right-skewed distribution with a small cluster of high-value observations beyond the upper whisker, confirming the presence of outliers.
- The concentration of extreme values is limited to **TotalCharges**, while the other numeric features remain within expected ranges.
- These results support the application of outlier treatment to **TotalCharges** only, while leaving **tenure** and **MonthlyCharges** unchanged.

After IQR capping – descriptive stats:			
	tenure	MonthlyCharges	TotalCharges
count	12051.00	12051.00	12051.00
mean	31.39	64.68	2181.90
std	24.18	29.74	2159.15
min	0.00	15.29	0.00
25%	8.00	35.45	505.95
50%	28.00	71.38	1333.33
75%	53.00	88.80	3548.02
max	74.00	121.67	8111.11

Figure 12: Descriptive Statistics After IQR Capping

- Following duplicate removal and outlier treatment, the dataset contains **12,051 records** with no missing values or duplicate observations.
- Outlier capping was applied only to **TotalCharges**, reducing the influence of extreme high-value observations while preserving all records.
- The maximum value of **TotalCharges** was successfully capped at the calculated IQR upper bound (**\$8,111.11**), confirming that the winsorization process was implemented correctly.
- Summary statistics for **tenure** and **MonthlyCharges** remained largely unchanged, indicating that these variables were unaffected by the outlier treatment.
- Overall, the dataset is fully cleaned and ready for feature engineering, exploratory analysis, and predictive modelling.

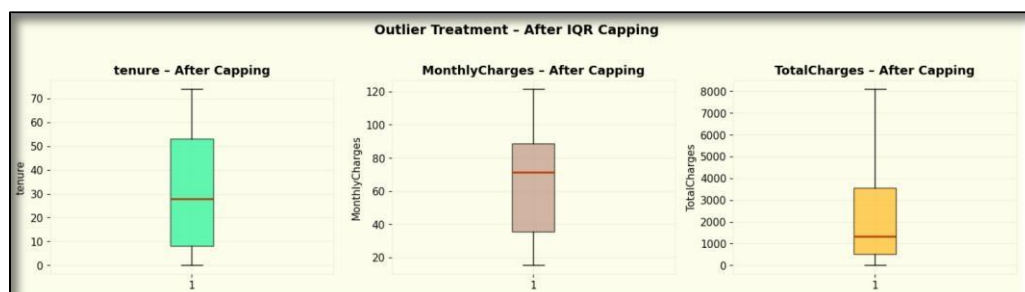


Figure 13: Boxplots of Numeric Features After IQR Capping

- **Tenure** and **MonthlyCharges** remained unchanged after outlier treatment, confirming that no outliers were detected in these variables.
- For **TotalCharges**, the extreme values observed before treatment were successfully capped, eliminating the outlier cluster while preserving the overall distribution.
- The median and interquartile range remained largely unaffected, indicating that the treatment impacted only a small number of extreme observations.
- The before-and-after comparison confirms that outlier treatment was applied selectively and effectively, improving data quality without materially altering the underlying distribution.

Final shape: (12051, 21)
Overall churn rate: 28.2%

Figure 14: Final Cleaned Dataset Shape and Overall Churn Rate

- The final dataset contains **12,051 records and 21 features**, reflecting duplicate removal and the addition of the binary target variable (**Churn_bin**).
- The churn rate remains **28.2%**, indicating that approximately one in four customers discontinued the service.
- The target distribution reveals a moderate class imbalance, with non-churners significantly outnumbering churners.
- Following data cleaning, missing-value treatment, outlier handling, and target encoding, the dataset is fully prepared for feature engineering, exploratory analysis, and predictive modelling.

5. Exploratory Data Analysis.

5.1. Target Distributions.

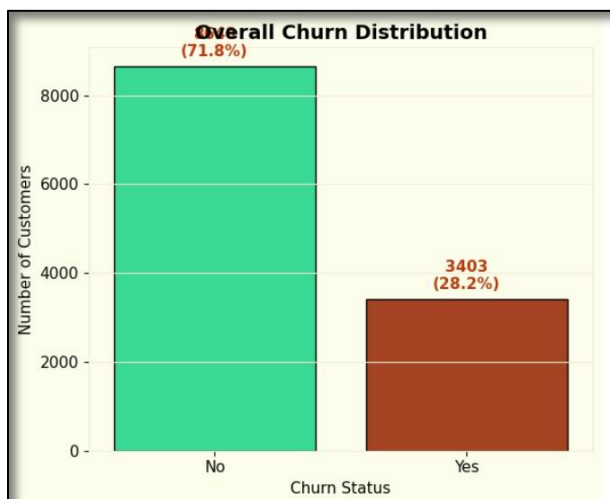


Figure 15: Overall Churn Distribution

- The dataset contains **8,648 non-churners (71.8%)** and **3,403 churners (28.2%)**, indicating a moderate class imbalance.
- A churn rate of **28.2%** is substantial from a business perspective, suggesting that more than one in four customers discontinue the service.
- The imbalance highlights the need for appropriate class-balancing techniques and evaluation metrics during model development.
- The overall churn rate serves as a benchmark for subsequent analyses, with segments exceeding this rate representing higher-risk customer groups.

5.2. Numeric Features.

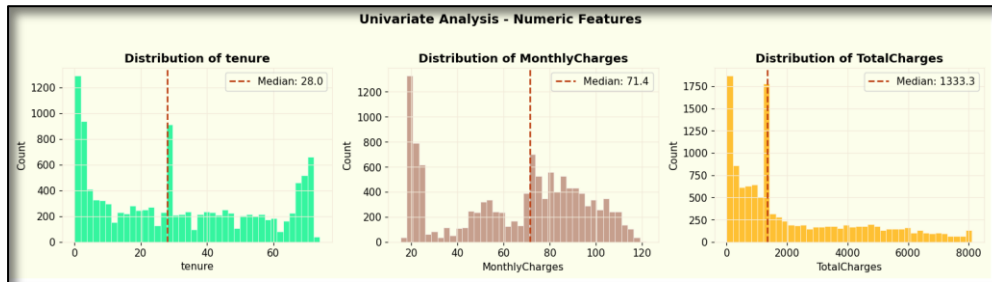


Figure 16: Distribution of Numeric Features

- **tenure** exhibits a right-skewed and partially bimodal distribution, with a large concentration of new customers and a smaller group of long-tenured customers, indicating distinct customer lifecycle segments.
- **MonthlyCharges** shows evidence of multiple pricing tiers, with one cluster of lower-cost plans and another representing customers subscribed to higher-value bundled services.
- **TotalCharges** is strongly right-skewed, with most customers concentrated at lower cumulative spending levels and a smaller group of high-value, long-tenured customers.
- The concentration of customers with low tenure and low cumulative spending highlights a sizeable early-lifecycle segment that may be particularly vulnerable to churn.

5.3. Categorical Features.

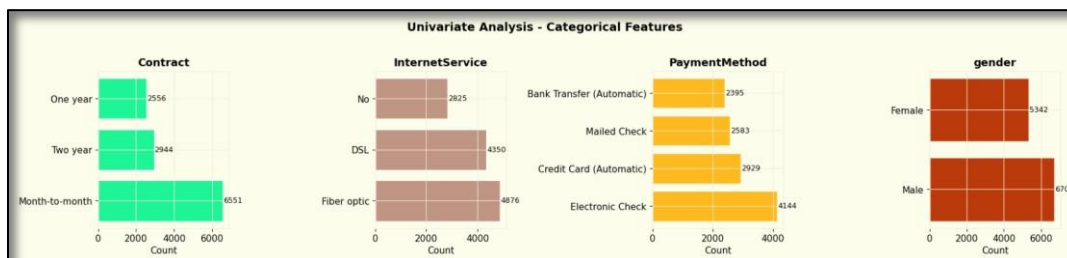


Figure 17: Distribution of Categorical Features

- **Month-to-month contracts** represent the largest customer segment, accounting for more than half of the customer base, indicating a substantial population with lower contractual commitment.
- **Fibre optic** is the most widely used internet service, making it a strategically important segment for churn management due to its large customer base.
- **Electronic Check** is the most common payment method and warrants further attention given its association with higher churn levels observed in later analyses.
- The gender distribution is relatively balanced, suggesting that gender alone is unlikely to be a primary driver of churn.
- From a business perspective, customers combining **month-to-month contracts**, **fibre optic service**, and **electronic check payments** may represent a high-priority retention segment due to the overlap of high-volume and high-risk characteristics.

5.4. Churn by Contract Type.

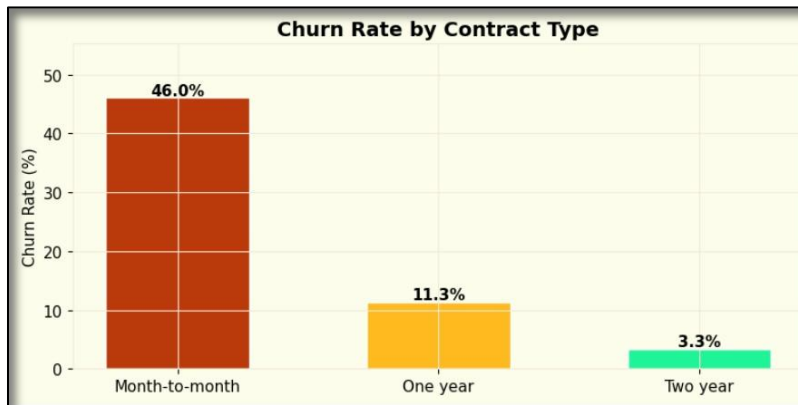


Figure 18: Churn Rate by Contract Type

- Churn rates vary substantially by contract type, with **Month-to-month customers exhibiting the highest churn**, followed by **One-year**, while **Two-year contracts show the lowest churn levels**.
- This indicates that **contract type is one of the strongest predictors of customer churn** in the dataset.
- Given that Month-to-month contracts represent the largest customer segment, improvements in retention within this group could significantly reduce overall churn.
- The consistent decline in churn as contract duration increases suggests that longer-term customer commitments are associated with stronger retention and customer loyalty.

5.5. Tenure by Churn Status.

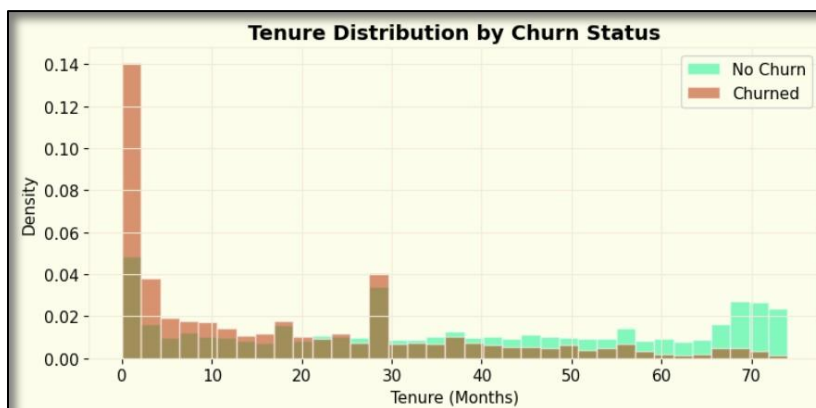


Figure 19: Tenure Distribution by Churn Status

- Churned customers are heavily concentrated in the **early months of tenure**, indicating that the risk of churn is highest shortly after customer acquisition.
- As tenure increases, the proportion of non-churning customers rises significantly, suggesting that long-tenured customers are substantially more likely to remain with the company.
- The relationship between tenure and churn highlights a clear retention pattern, where customer loyalty strengthens over time.
- These findings suggest that the greatest opportunity for churn reduction lies in improving customer onboarding and engagement during the initial months of the customer lifecycle.

5.6. Charges vs Churn.

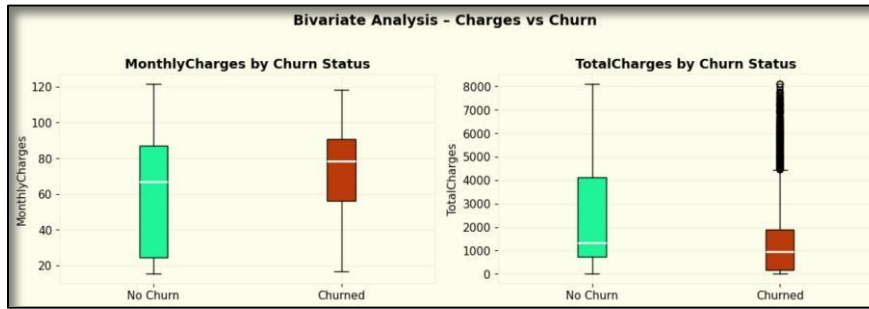


Figure 20: Monthly Charges and Total Charges by Churn Status

- Churned customers exhibit **higher monthly charges** than non-churners, suggesting that customers on more expensive plans are more likely to leave.
- In contrast, churned customers have **lower total charges**, indicating that many leave before accumulating substantial lifetime value.
- Together, these findings suggest that churn is concentrated among **relatively new customers paying higher monthly fees**, reinforcing the relationship between low tenure, high monthly charges, and elevated churn risk.
- From a business perspective, improving value perception and engagement among higher-paying customers early in their lifecycle could help reduce churn.
- A small number of high-value, long-tenured customers also churned, indicating the presence of additional retention factors that may warrant further investigation.

5.7. Churn Rate Across Categorical Features.

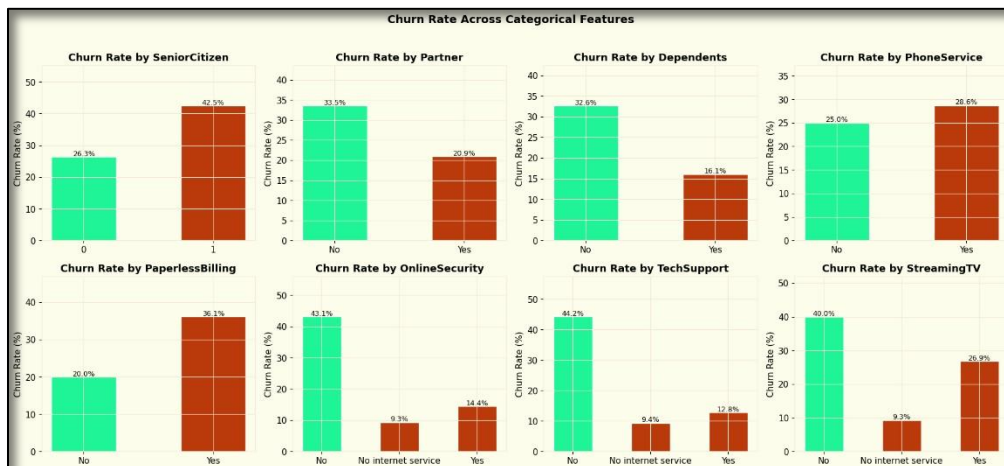


Figure 21: Churn Rate Across Categorical Features

- Significant differences in churn rates are observed across both demographic and service-related variables.
- Among demographic factors, **SeniorCitizen**, **Partner**, and **Dependents** show strong relationships with churn, with senior customers and those without partners or dependents exhibiting higher churn rates.
- **PaperlessBilling** is also associated with elevated churn, suggesting that digitally engaged customers may be more likely to switch providers.
- Service-related features show some of the strongest effects. Customers without **OnlineSecurity** and **TechSupport** experience substantially higher churn rates than those who subscribe to these services.
- **StreamingTV** is associated with lower churn, although its impact appears weaker than that of OnlineSecurity and TechSupport.

- From a business perspective, promoting value-added services such as **OnlineSecurity** and **TechSupport** may offer a high-impact retention strategy. At the same time, vulnerable customer segments can be targeted through tailored engagement and support programs.

5.8. Churn by Internet Service.

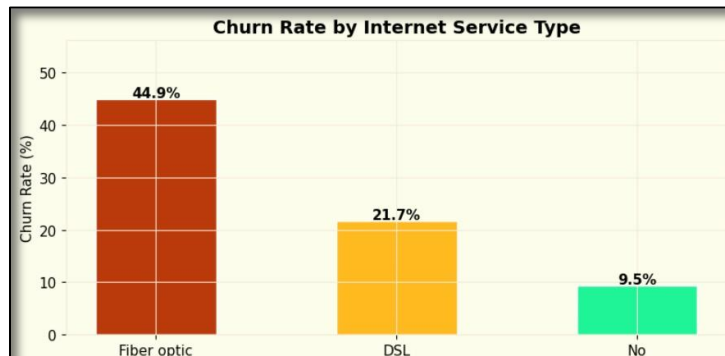


Figure 22: Churn Rate by Internet Service Type

- Fibre optic customers exhibit the highest churn rate** among all internet service categories, substantially exceeding the overall churn rate.
- DSL customers** experience lower churn levels, while customers with **no internet service** show the strongest retention.
- The variation in churn across internet service types suggests that service experience, pricing, and perceived value may play important roles in customer retention.
- From a business perspective, the high churn rate among fibre optic customers is particularly concerning, as this segment likely contributes significantly to revenue. Improving service quality, customer experience, and value perception within this segment could have a meaningful impact on overall retention.

5.9. Churn by Payment Method.

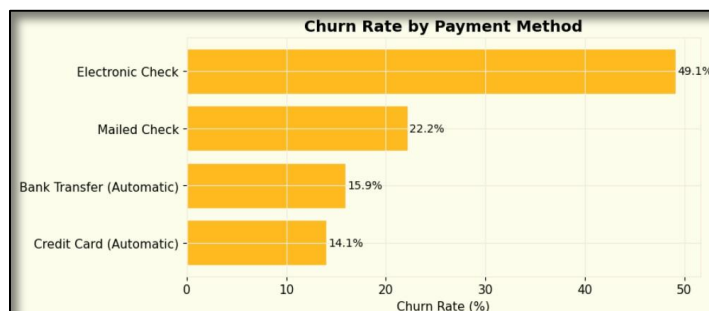


Figure 23: Churn Rate by Payment Method

- Electronic Check customers exhibit the highest churn rate** among all payment methods, substantially exceeding the overall churn rate.
- In contrast, customers using **automatic payment methods** demonstrate significantly lower churn levels, indicating a strong association between auto-pay adoption and customer retention.
- Given that Electronic Check is also one of the largest customer segments, its elevated churn rate represents a meaningful business risk.
- The results suggest that encouraging customers to adopt automatic payment methods could serve as an effective retention strategy and help reduce overall churn.

5.10. Multivariate (Correlation Heatmap)

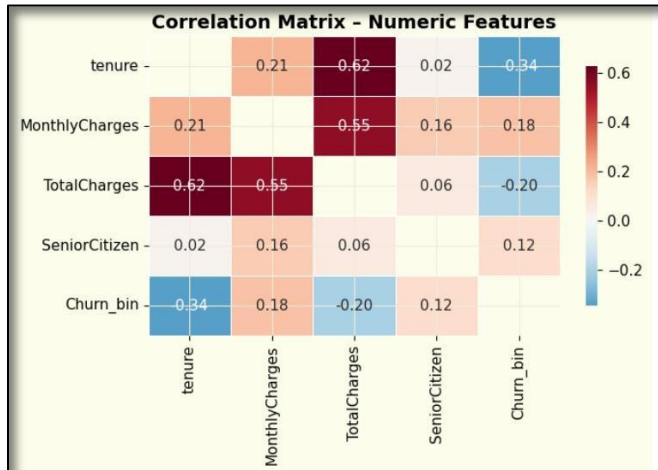


Figure 24: Correlation Matrix of Numeric Features

- The strongest positive relationship is observed between **tenure** and **TotalCharges**, reflecting the natural accumulation of customer spending over time.
- **MonthlyCharges** also shows a positive association with **TotalCharges**, indicating that customers on higher-priced plans tend to generate greater lifetime revenue.
- **tenure** exhibits the strongest relationship with **churn**, with longer-tenured customers being less likely to leave the service.
- **MonthlyCharges** is positively associated with churn, suggesting that customers paying higher monthly fees are somewhat more likely to churn.
- The correlations support earlier findings that customer retention improves with tenure, while higher monthly costs may contribute to churn risk.
- No severe multicollinearity is observed among the predictor variables, indicating that all key features can be retained for model development.

5.11. KDE Plot.

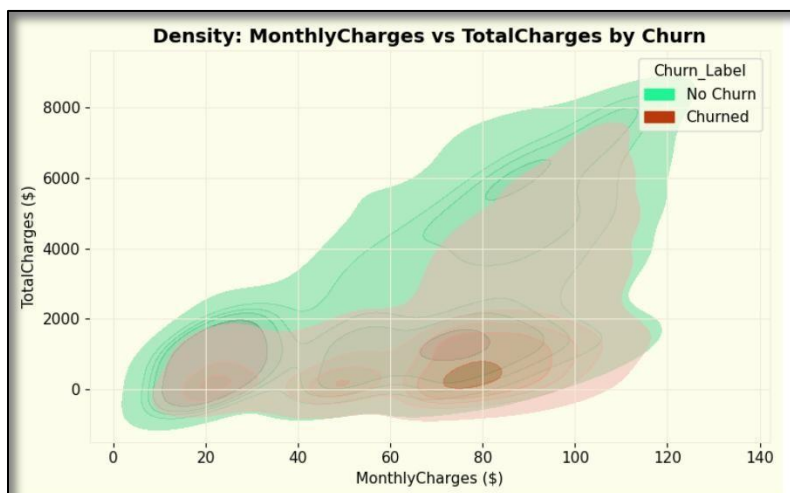


Figure 25: Density of Monthly Charges vs Total Charges by Churn Status

- Non-churning customers are distributed across a broad range of **MonthlyCharges** and **TotalCharges**, with many concentrated in higher cumulative spending segments.
- Churned customers are primarily concentrated in **low TotalCharges** regions, indicating that many leave before generating substantial lifetime value.

- A notable churn cluster consists of customers with **high MonthlyCharges but low TotalCharges**, suggesting that relatively new customers on premium-priced plans face elevated churn risk.
- Customers with high **TotalCharges** are predominantly retained, reinforcing earlier findings that churn likelihood declines as customer tenure and accumulated value increase.
- Overall, the combination of **high MonthlyCharges and low TotalCharges** emerges as a key risk profile that should be prioritised for retention efforts.

6. Feature Engineering & Final Preprocessing

```
Residual NaN cells after encoding : 0
Rows containing at least one NaN : 0
That is 0.00% of the dataset – safe to drop
Shape after drop: (12051, 35)
Class distribution:
Churn_bin
0      8648
1      3403
Name: count, dtype: int64
```

Figure 26: Post-Encoding NaN Check and Class Distribution

- Two new features were engineered to enhance predictive power: **TenureGroup**, which captures customer lifecycle stages, and **ChargePerMonth**, which reflects average customer spending over time.
- Categorical variables were transformed using **one-hot encoding**, converting all model inputs into a machine-learning-ready format while avoiding multicollinearity.
- The encoding process expanded the dataset to **35 features** without introducing any missing values.
- The final dataset contains **12,051 records and 35 model-ready features**, with the original class distribution preserved.
- Following feature engineering and encoding, the dataset was fully prepared for train-test splitting, class balancing, scaling, and model development.

7. Class Imbalance & SMOTE

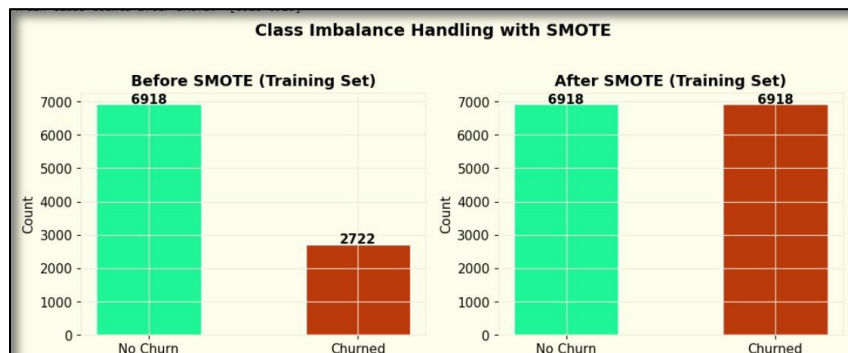


Figure 27: Class Distribution Before and After SMOTE

- The dataset was divided into **training and testing sets using stratified sampling**, ensuring that the original churn distribution was preserved across both subsets.
- The training data exhibited a moderate class imbalance, with non-churners substantially outnumbering churners.
- To address this imbalance, **SMOTE** was applied to the training set, creating a balanced class distribution and improving the model's ability to learn churn-related patterns.
- The test set was intentionally left unchanged to ensure that model evaluation reflects real-world customer distributions.
- Applying SMOTE only to the training data

8. Feature Scaling.

```
scaler = StandardScaler()
X_res_sc = scaler.fit_transform(X_res) # fit ONLY on training data
X_test_sc = scaler.transform(X_test) # apply same scale to test
```

Figure 28: Feature Scaling Using StandardScaler

- **StandardScaler** was applied to standardise feature values, ensuring that all variables operate on a comparable scale and preventing features with larger magnitudes from disproportionately influencing model performance.
- Scaling parameters were learned exclusively from the training data and subsequently applied to the test set, preserving the integrity of the evaluation process.
- This approach prevents data leakage and ensures that model performance reflects realistic deployment conditions.
- Following scaling, the data was fully prepared for model development and evaluation.

9. Baseline Model.

Methodology

Logistic Regression was selected as the baseline model due to its simplicity, interpretability, and suitability as a linear benchmark against which all advanced models are compared. The model was trained on the SMOTE-balanced, StandardScaler-normalised training set and evaluated on the heldout test set using five metrics: Accuracy, Precision, Recall, F1 Score, and ROC-AUC.

ROC-AUC was chosen as the primary evaluation metric for the following reasons:

- It measures the model's ability to discriminate between churners and non-churners across all classification thresholds, making it robust to class imbalance
- A high ROC-AUC means the model ranks a randomly chosen churner above a randomly chosen non-churner with high probability — directly relevant to prioritising retention outreach
- Unlike Accuracy, it does not give misleading results when one class significantly outnumbers the other, as is the case here with a 72:28 split

Metric	Why Considered	Selected as Primary?
Accuracy	Simple but misleading under class imbalance	✗ No
Precision	Measures false alarm rate	✗ No
Recall	Measures missed churners	✗ No
F1 Score	Balances Precision and Recall	✗ No
ROC-AUC	Threshold-independent, robust to imbalance	☑ Yes

Table 2: Evaluation Metrics Considered and Headline Metric Selection

Logistic Regression			
Metric	Train	Test	Overfit?
Accuracy	0.8130	0.7715	OK
Precision	0.8123	0.5827	Overfit
Recall	0.8141	0.6725	Overfit
F1	0.8132	0.6244	Overfit
ROC-AUC	0.9010	0.8225	Overfit
Test Classification Report:			
	precision	recall	f1-score support
No Churn	0.86	0.81	0.84 1730
Churned	0.58	0.67	0.62 681
accuracy			0.77 2411
macro avg	0.72	0.74	0.73 2411
weighted avg	0.78	0.77	0.78 2411

Figure 29: Logistic Regression Baseline Performance Metrics

Metric	Train	Test	Gap	Overfit?
Accuracy	0.8130	0.7715	0.0415	OK
Precision	0.8123	0.5827	0.2296	Yes
Recall	0.8141	0.6725	0.1416	Yes
F1 Score	0.8132	0.6244	0.1888	Yes
ROC-AUC	0.9010	0.8225	0.0785	Marginal

Table 3: Train vs Test Overfitting Check

Model Performance Observations:

- The baseline achieved a **Test ROC-AUC of 0.8225**, establishing a strong benchmark that all advanced models must exceed to justify their added complexity.
- Accuracy (Test = 0.7715)** shows a small train-test gap of 0.04, indicating stable overall classification with no severe overfitting on the majority class.
- Precision (Test = 0.5827)** means that of all customers flagged as likely churners, approximately 58% are actual churners — 42% of retention efforts would be directed at customers who were not going to leave.
- Recall (Test = 0.6725)** means the model correctly identifies 67% of actual churners, missing approximately 33%. These missed churners leave without any retention intervention, each representing a potential loss of \$857 in annual recurring revenue.
- F1 Score (Test = 0.6244)** reflects the trade-off between Precision and Recall. The lower test F1 compared to training (0.8132) confirms that SMOTE-generated synthetic samples inflated training performance — expected behaviour when oversampling is applied only to training data.
- ROC-AUC gap of 0.079** between train and test is marginal but acceptable for a linear model. Logistic Regression's inability to capture non-linear interactions between features such as contract type, tenure group, and payment method limits its discriminative power and motivates exploration of advanced models.

Confusion Matrix Summary — Logistic Regression

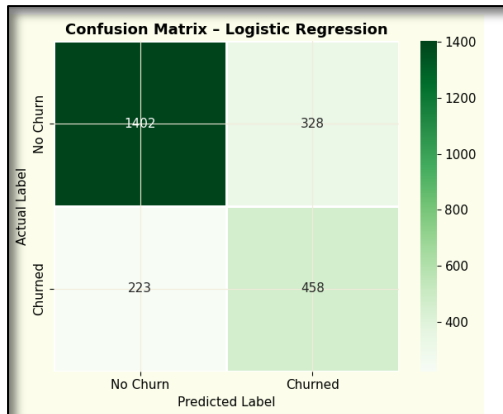


Figure 30: Confusion Matrix — Logistic Regression

	Predicted No Churn	Predicted Churned
Actual No Churn	1,402 (TN)	328 (FP)
Actual Churned	223 (FN)	458 (TP)

Table 4: Confusion Matrix Summary — Logistic Regression

- **223 missed churners (false negatives)** represent approximately **\$191,111 in unrecovered annual revenue** at \$857 per customer — the primary business cost of the baseline model.
- **328 incorrectly flagged non-churners (false positives)** result in approximately **\$9,840 in unnecessary retention spend** at \$30 per intervention — significantly lower than the cost of missed churners, confirming that a higher-recall model is preferable for this use case.
- The confusion matrix indicates that the model correctly classified the majority of both churning and non-churning customers.
- Most prediction errors arise from **false negatives**, where customers likely to churn are incorrectly classified as non-churners.
- From a business perspective, false negatives are the most costly errors because these customers leave without receiving any retention intervention.
- Although some false positives are generated, these primarily result in additional retention efforts and are generally less costly than missed churners.
- Overall, the baseline model provides a useful foundation for churn prediction, but reducing missed churners remains a key objective for subsequent advanced models.

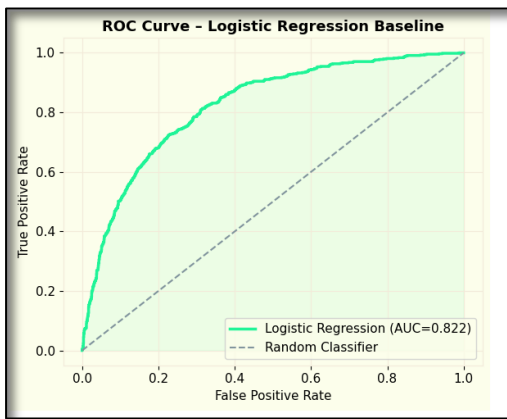


Figure 31: ROC Curve — Logistic Regression Baseline

- The baseline Logistic Regression model achieved an **ROC-AUC of 0.822**, indicating strong discrimination between churners and non-churners.
- The ROC curve remains substantially above the random-classification baseline, confirming that the model can effectively distinguish between the two classes.
- The model demonstrates a favourable balance between identifying churners and limiting false positive predictions across different classification thresholds.
- From a business perspective, the model provides a reliable foundation for prioritising customers based on churn risk and establishes a strong benchmark for evaluating advanced models.
- The steep initial rise of the ROC curve indicates the model can identify a large proportion of churners at very low false positive rates — useful for high-precision retention campaigns with limited budgets.

The classification threshold can be adjusted by the business depending on operational priorities: lowering the threshold increases churner detection (Recall) at the cost of more false alarms, while raising it improves Precision but misses more churners.

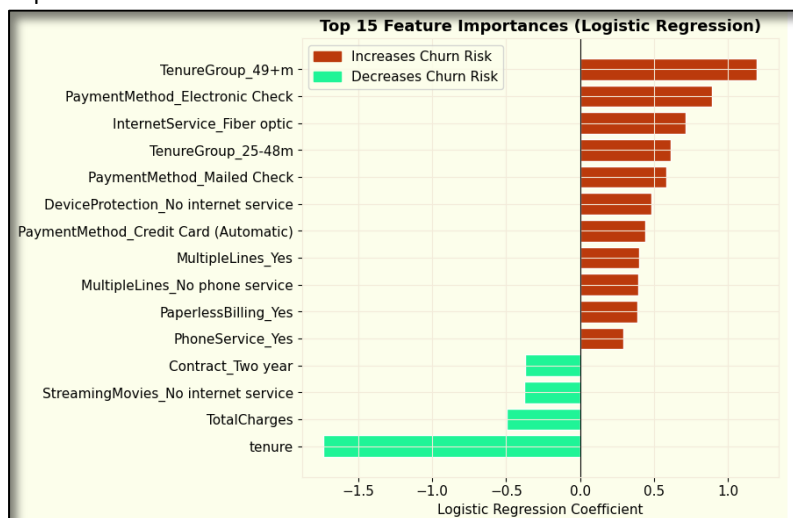


Figure 32: Top 15 Feature Importances (Logistic Regression)

- **Tenure** emerged as the strongest predictor of churn, with longer-tenured customers demonstrating a substantially lower likelihood of leaving the service.
- **TotalCharges** and **Two-year contracts** were also associated with reduced churn risk, highlighting the importance of long-term customer relationships and commitment.
- Among churn-increasing factors, **Electronic Check** and **Fibre optic service** exhibited some of the strongest positive effects, indicating elevated churn risk within these customer segments.
- Several service-related features showed smaller positive associations with churn, suggesting that customer behaviour and service adoption patterns also contribute to churn outcomes.

- Overall, the model confirms that **customer tenure, contract type, payment method, and internet service type** are among the most influential drivers of churn and should be prioritised in retention strategies.

Baseline Model Limitations and Motivation for Advanced Models

- Logistic Regression assumes a **linear decision boundary**, which cannot capture complex interactions between features such as the combined effect of short tenure, high monthly charges, and electronic check payment simultaneously.
- The **Precision-Recall trade-off** remains suboptimal at the default threshold — the model either misses too many churners or flags too many non-churners depending on the threshold chosen, limiting its operational flexibility.
- The **train-test gap on Precision (0.23) and Recall (0.14)** suggests the model struggles to generalise minority class predictions to unseen data, a known limitation of linear models when applied to imbalanced datasets even after SMOTE balancing.
- These limitations directly motivate the exploration of non-linear ensemble models in Section 10, which can capture feature interactions, handle imbalanced patterns more effectively, and produce better-calibrated churn probability scores for business deployment.

10. Advanced Models.

Decision Tree				
Metric	Train	Test	Overfit?	
Accuracy	0.9993	0.7192	Overfit	
Precision	1.0000	0.5027	Overfit	
Recall	0.9986	0.5404	Overfit	
F1	0.9993	0.5209	Overfit	
ROC-AUC	1.0000	0.6650	Overfit	
Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.81	0.79	0.80	1730
Churned	0.50	0.54	0.52	681
accuracy			0.72	2411
macro avg	0.66	0.66	0.66	2411
weighted avg	0.73	0.72	0.72	2411

Figure 33: Logistic Regression Metrics Summary

- The Decision Tree exhibited **severe overfitting**, achieving near-perfect performance on the training data but substantially weaker results on the test set.
- Despite its excellent training performance, the model produced the **lowest test ROC-AUC** among the evaluated models, indicating poor generalisation to unseen data.
- The model's churn detection performance was also weaker than the Logistic Regression baseline, resulting in a higher proportion of missed churners.
- Overall, the results suggest that the unpruned Decision Tree memorised training patterns rather than learning robust relationships applicable to new customers.
- Given its poor generalisation performance, the Decision Tree was not considered a suitable candidate for deployment and was excluded from further tuning.

Bagging				
Metric	Train	Test	Overfit?	
Accuracy	0.9993	0.7698	Overfit	
Precision	0.9988	0.5960	Overfit	
Recall	0.9997	0.5742	Overfit	
F1	0.9993	0.5849	Overfit	
ROC-AUC	1.0000	0.8083	Overfit	
Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.83	0.85	0.84	1730
Churned	0.60	0.57	0.58	681
accuracy			0.77	2411
macro avg	0.72	0.71	0.71	2411
weighted avg	0.77	0.77	0.77	2411

Figure 34: Bagging Classifier Metrics Summary

- The Bagging model reduced overfitting compared to the standalone Decision Tree, demonstrating the benefits of ensemble learning for improving model stability and generalisation.
- Test performance improved substantially over the Decision Tree and remained competitive with the Logistic Regression baseline.
- While overall predictive performance was satisfactory, the model identified fewer churners than the baseline model, limiting its effectiveness for retention-focused use cases.
The model performed particularly well in identifying non-churning customers but was less effective at detecting customers at risk of leaving.
- Overall, Bagging provided a meaningful improvement over a single Decision Tree but did not offer sufficient advantages over higher-performing ensemble methods to justify further tuning

Random Forest				
Metric	Train	Test	Overfit?	
Accuracy	0.9993	0.7702	Overfit	
Precision	0.9990	0.5941	Overfit	
Recall	0.9996	0.5888	Overfit	
F1	0.9993	0.5914	Overfit	
ROC-AUC	1.0000	0.8209	Overfit	
Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.84	0.84	0.84	1730
Churned	0.59	0.59	0.59	681
accuracy			0.77	2411
macro avg	0.72	0.72	0.72	2411
weighted avg	0.77	0.77	0.77	2411

Figure 35: Random Forest Metrics Summary

- The Random Forest model demonstrated strong predictive performance and generalised substantially better than a standalone Decision Tree.
- Among the non-boosting ensemble methods, Random Forest achieved the strongest overall test performance, with results comparable to the Logistic Regression baseline.
- Although the model showed balanced performance across classes, its ability to identify churners remained slightly below the baseline model.
- The results indicate that Random Forest has strong predictive potential but may require parameter optimisation to leverage its capabilities fully.
- Given its competitive performance and scope for improvement, Random Forest was selected for hyperparameter tuning.

AdaBoost				
Metric	Train	Test	Overfit?	
Accuracy	0.8144	0.7710	OK	
Precision	0.8078	0.5780	Overfit	
Recall	0.8251	0.7019	Overfit	
F1	0.8164	0.6340	Overfit	
ROC-AUC	0.9012	0.8253	Overfit	
Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.87	0.80	0.83	1730
Churned	0.58	0.70	0.63	681
accuracy			0.77	2411
macro avg	0.72	0.75	0.73	2411
weighted avg	0.79	0.77	0.78	2411

Figure 36: AdaBoost Metrics Summary

- AdaBoost demonstrated the strongest generalisation performance among the models evaluated so far, with minimal evidence of overfitting.
- The model achieved a higher test ROC-AUC than the Logistic Regression baseline, indicating improved discrimination between churners and non-churners.
- AdaBoost was particularly effective at identifying churners, achieving the highest recall among the evaluated models and reducing the risk of missed churners.
- This improvement came with a modest increase in false positives, reflecting the typical precision–recall trade-off in churn prediction.
- Overall, AdaBoost delivered a strong balance between predictive performance and generalisation, making it one of the top-performing advanced models in the study.

Although its performance was competitive, Gradient Boosting and XGBoost were prioritised hyperparameter tuning due to their stronger overall potential for further improvement.

Gradient Boosting			
Metric	Train	Test	Overfit?
Accuracy	0.8406	0.7793	Overfit
Precision	0.8360	0.5971	Overfit
Recall	0.8475	0.6725	Overfit
F1	0.8417	0.6326	Overfit
ROC-AUC	0.9229	0.8358	Overfit

Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.86	0.82	0.84	1730
Churned	0.60	0.67	0.63	681
accuracy			0.78	2411
macro avg	0.73	0.75	0.74	2411
weighted avg	0.79	0.78	0.78	2411

Figure 37: Gradient Boosting Metrics Summary

- Gradient Boosting achieved the strongest test performance among the models evaluated, demonstrating excellent discrimination between churners and non-churners.
- While some overfitting was observed, the train-test performance gap remained moderate and substantially lower than that of the tree-based bagging models.
- The model maintained strong churn detection capability while improving overall predictive performance compared to the Logistic Regression baseline.
- Performance was balanced across both churning and non-churning customers, indicating robust generalisation to unseen data.
- Overall, Gradient Boosting emerged as one of the top-performing models and was selected for hyperparameter tuning due to its strong predictive capability and potential for further improvement.

XGBoost			
Metric	Train	Test	Overfit?
Accuracy	0.9342	0.7769	Overfit
Precision	0.9294	0.6032	Overfit
Recall	0.9399	0.6138	Overfit
F1	0.9346	0.6084	Overfit
ROC-AUC	0.9849	0.8228	Overfit

Test Classification Report:			
	precision	recall	f1-score
No Churn	0.85	0.84	0.84
Churned	0.60	0.61	0.61

accuracy			0.78
macro avg	0.73	0.73	0.73
weighted avg	0.78	0.78	0.78

Figure 38: XGBoost Metrics Summary

- XGBoost demonstrated strong learning capability but showed clear signs of overfitting, with performance on the training set substantially exceeding that on the test set.
 - Despite its strong training performance, the untuned model delivered test results comparable to the Logistic Regression baseline and below the best-performing boosting models.
 - The model was less effective at identifying churners than AdaBoost and Gradient Boosting, indicating that overfitting negatively impacted its ability to generalise to unseen customers.
 - Nevertheless, XGBoost maintained strong overall predictive performance and showed considerable potential for improvement through appropriate regularisation and parameter optimisation.
- Given its strong underlying learning capacity and scope for performance gains, XGBoost was selected for hyperparameter tuning.

=== ALL MODELS - TRAIN vs TEST SUMMARY ===											
Model	Train Accuracy	Test Accuracy	Train Precision	Test Precision	Train Recall	Test Recall	Train F1	Test F1	Train ROC-AUC	Test ROC-AUC	
Logistic Regression	0.8130	0.7715	0.8123	0.5827	0.8141	0.6725	0.8132	0.6244	0.9810	0.8225	
Decision Tree	0.9993	0.7192	1.0000	0.5027	0.9986	0.5404	0.9993	0.5209	1.0000	0.6650	
Bagging	0.9993	0.7698	0.9988	0.5960	0.9997	0.5742	0.9993	0.5849	1.0000	0.8083	
Random Forest	0.9993	0.7702	0.9990	0.5941	0.9996	0.5888	0.9993	0.5914	1.0000	0.8209	
AdaBoost	0.8144	0.7710	0.8078	0.5780	0.8251	0.7019	0.8164	0.6340	0.9012	0.8253	
Gradient Boosting	0.8406	0.7793	0.8360	0.5971	0.8475	0.6725	0.8417	0.6326	0.9229	0.8358	
XGBoost	0.9342	0.7769	0.9294	0.6032	0.9399	0.6138	0.9346	0.6084	0.9849	0.8228	

Figure 39: All Models — Train vs Test Performance Summary

- The model comparison highlights clear differences in predictive performance and generalisation across the evaluated algorithms.
- **Gradient Boosting** achieved the strongest overall performance on the primary evaluation metric (ROCAUC), while **AdaBoost** demonstrated the highest ability to identify churners.
- **Decision Tree** was the weakest performer due to severe overfitting, resulting in poor generalisation to unseen data despite excellent training performance.
- Ensemble methods, particularly boosting-based models, consistently outperformed the standalone Decision Tree and showed stronger generalisation than bagging-based approaches.
- Logistic Regression remained a competitive benchmark, demonstrating that a simple, interpretable model can achieve performance comparable to several advanced algorithms.
- Based on overall performance, generalisation ability, and potential for further improvement, **Random Forest, Gradient Boosting, and XGBoost** were selected for hyperparameter tuning.

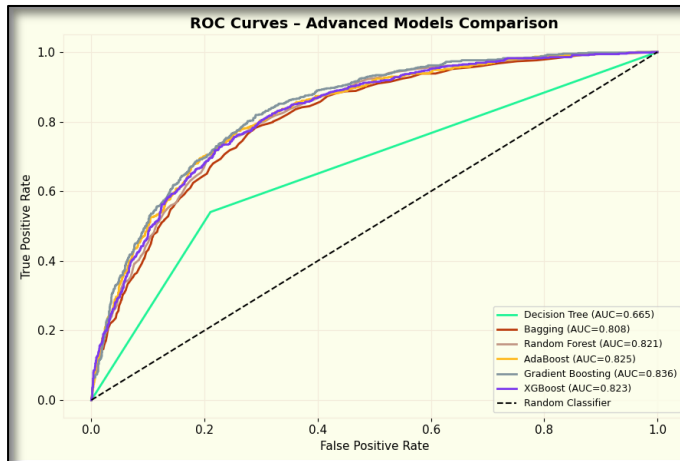


Figure 40: ROC Curves — Advanced Models Comparison

- The ROC curves highlight clear differences in model discrimination ability across the evaluated algorithms.
- **Gradient Boosting** achieved the strongest overall performance, with its ROC curve consistently dominating the other models across most classification thresholds.
- **Decision Tree** was the weakest performer, exhibiting substantially poorer discrimination and confirming the overfitting issues identified during model evaluation.
- The remaining ensemble models demonstrated broadly similar performance, with only modest differences in their ability to distinguish churners from non-churners.
- The strong separation of the top-performing curves from the random-classification baseline confirms that these models provide meaningful predictive value for churn prediction.

Based on overall ROC performance and model behaviour, **Random Forest, Gradient Boosting, and XGBoost** were selected for hyperparameter tuning.

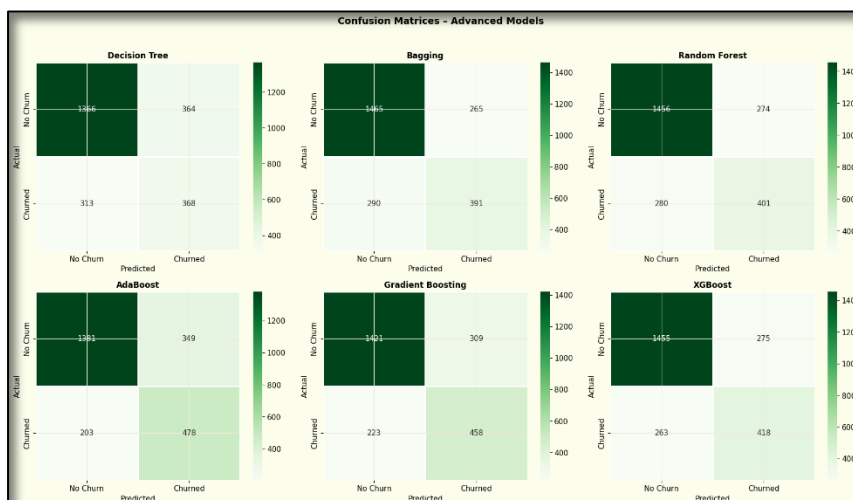


Figure 41: Confusion Matrices — Advanced Models Comparison

-
- The confusion matrix comparison highlights important differences in how effectively each model identifies churners while controlling prediction errors.
- **AdaBoost** detected the highest number of churners, demonstrating strong sensitivity to at-risk customers.
- **Gradient Boosting** achieved the most balanced performance, maintaining strong churn detection while limiting both missed churners and unnecessary retention interventions.
- **Decision Tree** performed the weakest overall, generating comparatively high levels of both missed churners and incorrect churn predictions, further confirming its poor generalisation.
- The results indicate that boosting-based models provide a more favourable balance between churn detection and prediction accuracy than standalone or bagging-based tree models.
- From a business perspective, **Gradient Boosting** offers the most practical balance between maximising retention opportunities and controlling retention costs, supporting its selection for further optimisation.

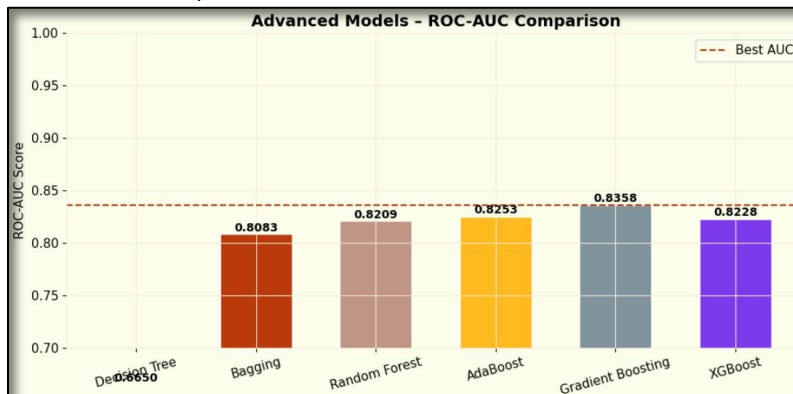


Figure 42: Advanced Models — ROC-AUC Comparison

- The confusion matrix comparison reveals meaningful differences in how effectively the models identify churners while minimising prediction errors.
- **AdaBoost** demonstrated the strongest churn detection capability, successfully identifying the highest proportion of at-risk customers.
- **Gradient Boosting** delivered the most balanced performance, achieving strong churn detection while maintaining a reasonable level of false predictions.
- **Decision Tree** performed the weakest overall, further confirming the overfitting issues observed in earlier evaluations.
- Overall, boosting-based models showed superior performance compared to standalone and baggingbased tree models, offering a better balance between customer retention opportunities and operational efficiency.
- Based on its balanced error profile and strong predictive performance, **Gradient Boosting** emerged as the most suitable model for further optimisation and final model selection.



Figure 43: Train vs Test ROC-AUC — All Models

- The train-test ROC-AUC comparison highlights substantial differences in model generalisation across the evaluated algorithms.
- **Decision Tree, Bagging, and Random Forest** exhibited the largest train-test performance gaps, indicating a tendency to overfit the training data.
- **XGBoost** also showed noticeable overfitting, suggesting that further regularisation and parameter optimisation could improve its generalisation performance.
- **Gradient Boosting** and **AdaBoost** demonstrated the most balanced train-test performance, indicating better control of overfitting while maintaining strong predictive capability.
- **Logistic Regression** remained a stable and competitive benchmark, achieving consistent performance despite its relatively simple structure.
- Overall, the results highlight the importance of hyperparameter tuning, particularly for models exhibiting larger train-test gaps, to improve reliability on unseen customer data.



Figure 43b: Train vs Test ROC-AUC — All Models

The Train-Test F1 comparison reveals notable differences in model generalisation performance across the evaluated algorithms.

Decision Tree, Bagging, and Random Forest exhibited the largest performance gaps, indicating substantial overfitting and reduced effectiveness on unseen data.

- **XGBoost** also showed a noticeable decline in test performance, suggesting that additional tuning is required to improve its generalisation capability.
- **Logistic Regression, AdaBoost, and Gradient Boosting** demonstrated more balanced train-test performance, indicating greater stability and reliability.
- Among the advanced models, **AdaBoost** and **Gradient Boosting** achieved the strongest balance between Precision and Recall on the test set.

-
-
- Overall, the F1-score results reinforce the earlier findings that boosting-based models generalise better than standalone and bagging-based tree models, supporting their selection for hyperparameter tuning and final model evaluation.

11. Hyperparameter Tuning.

Model Selection for Hyperparameter Tuning

Observations:

- **Random Forest, Gradient Boosting, and XGBoost** were selected for hyperparameter tuning based on their strong predictive performance and potential for further improvement.
- Random Forest demonstrated competitive test performance but showed signs of overfitting, indicating scope for improved generalisation through parameter optimisation.
- Gradient Boosting achieved the strongest overall performance among the evaluated models, making it a natural candidate for further refinement.
- XGBoost displayed strong learning capability but exhibited noticeable overfitting, suggesting that tuning could significantly enhance its performance on unseen data. **Tuning Methodology**

Observations:

- Hyperparameter optimisation was performed using **RandomizedSearchCV with 5-fold crossvalidation**, with **ROC-AUC** used as the optimisation metric.
- This approach enabled efficient exploration of multiple parameter combinations while maintaining computational efficiency.
- Cross-validation ensured that model performance was evaluated across multiple data partitions, improving the reliability and robustness of the tuning process.
- The tuning process aimed to improve model generalisation, reduce overfitting, and maximise predictive performance on unseen customer data.

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Fitting 5 folds for each of 20 candidates, totalling 100 fits
```

Figure 44: Hyperparameter Tuning — Cross-Validation Fitting Log

- Hyperparameter tuning was performed using **RandomizedSearchCV** on **Random Forest, Gradient Boosting, and XGBoost** models.
- Each model evaluated **20 random parameter combinations** using **5-fold cross-validation**, resulting in **100 model fits per algorithm**.
Cross-validation provides a more reliable estimate of model performance by testing each parameter set across multiple train-validation splits.
RandomizedSearchCV was selected over GridSearchCV because it offers efficient exploration of the hyperparameter space with lower computational cost.


```
Best RF params : {'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 'log2', 'max_depth': 15} | CV AUC: 0.9255
Best GB params : {'subsample': 0.7, 'n_estimators': 300, 'min_samples_split': 2, 'max_depth': 6, 'learning_rate': 0.05} | CV AUC: 0.9252
Best XGB params: {'subsample': 0.8, 'reg_alpha': 0, 'n_estimators': 300, 'max_depth': 6, 'learning_rate': 0.1, 'gamma': 0.3, 'colsample_bytree': 1.0} | CV AUC: 0.9266
```

Figure 45: Best Hyperparameters and Cross-Validated AUC by Model

- Random Forest achieved a CV ROC-AUC of **0.9255** with optimised tree depth and feature sampling, helping reduce overfitting.
- Gradient Boosting achieved a CV ROC-AUC of **0.9252**, using a lower learning rate and subsampling to improve generalisation.
- XGBoost achieved the highest CV ROC-AUC (**0.9266**), benefiting from regularisation and controlled tree growth.
- All three models produced very similar cross-validation performance (**ROC-AUC: 0.9252–0.9266**), indicating strong predictive capability.
- Final model selection was based on **test-set performance**, as cross-validation results alone do not guarantee better generalisation on unseen data.

12. Tuned Models Evaluation.

Random Forest (Tuned)				
Metric	Train	Test	Overfit?	
Accuracy	0.9610	0.7814	Overfit	
Precision	0.9429	0.6085	Overfit	
Recall	0.9815	0.6344	Overfit	
F1	0.9618	0.6211	Overfit	
ROC-AUC	0.9928	0.8336	Overfit	
Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.85	0.84	0.85	1730
Churned	0.61	0.63	0.62	681
accuracy			0.78	2411
macro avg	0.73	0.74	0.73	2411
weighted avg	0.78	0.78	0.78	2411

Figure 46: Random Forest (Tuned) Metrics Summary

- Hyperparameter tuning improved Random Forest performance, increasing Test ROC-AUC from 0.8209 to 0.8336 and reducing overfitting.
- Test Recall improved to 0.6344, enabling the model to identify more churners than the untuned version.
- Precision and F1-score for the churned class also improved, indicating better overall minority-class prediction.
- The train-test performance gap narrowed considerably, demonstrating improved model generalisation and stability.
- With a Test ROC-AUC of 0.8336, the tuned Random Forest became one of the strongest-performing models and a strong candidate for final model selection.

Gradient Boosting (Tuned)			
Metric	Train	Test	Overfit?
Accuracy	0.9157	0.7781	Overfit
Precision	0.9176	0.6061	Overfit
Recall	0.9133	0.6123	Overfit
F1	0.9155	0.6092	Overfit
ROC-AUC	0.9777	0.8286	Overfit

Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.85	0.84	0.85	1730
Churned	0.61	0.61	0.61	681
accuracy			0.78	2411
macro avg	0.73	0.73	0.73	2411
weighted avg	0.78	0.78	0.78	2411

Figure 47: Gradient Boosting (Tuned) Metrics Summary

- The tuned Gradient Boosting model achieved a Test ROC-AUC of 0.8286, slightly lower than the untuned model (0.8358), indicating that tuning did not improve predictive performance.
- Precision improved marginally, but Recall and F1-score for the churned class decreased, showing a trade-off between identifying churners and reducing false positives.
- Performance on the No Churn class remained largely unchanged, indicating that tuning mainly affected minority-class prediction.
- The tuned model demonstrated more stable generalisation with a smaller train-test performance gap.
- Overall, the untuned Gradient Boosting model remained the stronger performer and was preferred based on its superior test-set ROC-AUC and churn detection capability.

XGBoost (Tuned)				
Metric	Train	Test	Overfit?	
Accuracy	0.9404	0.7831	Overfit	
Precision	0.9392	0.6158	Overfit	
Recall	0.9419	0.6167	Overfit	
F1	0.9405	0.6163	Overfit	
ROC-AUC	0.9873	0.8247	Overfit	
Test Classification Report:				
	precision	recall	f1-score	support
No Churn	0.85	0.85	0.85	1730
Churned	0.62	0.62	0.62	681
accuracy			0.78	2411
macro avg	0.73	0.73	0.73	2411
weighted avg	0.78	0.78	0.78	2411

Figure 48: XGBoost (Tuned) Metrics Summary

- Hyperparameter tuning resulted in modest improvements for XGBoost, increasing Test ROC-AUC from 0.8228 to 0.8247.
- Precision, Recall, and F1-score for the churned class improved slightly, indicating better identification of churners while reducing false positives.
- The tuned model achieved the highest Precision (0.6158) and highest Accuracy (0.7831) among the tuned models.
- Performance on the No Churn class remained strong, with Precision, Recall, and F1-score all around 0.85.

- Although tuning improved overall performance, XGBoost's Test ROC-AUC remained below Tuned Random Forest (0.8336) and Untuned Gradient Boosting (0.8358), limiting its final ranking based on the primary evaluation metric.

=== TUNED MODELS - TRAIN vs TEST SUMMARY ===										
Model	Train Accuracy	Test Accuracy	Train Precision	Test Precision	Train Recall	Test Recall	Train F1	Test F1	Train ROC-AUC	Test ROC-AUC
Random Forest (Tuned)	0.9610	0.7814	0.9429	0.6085	0.9815	0.6344	0.9618	0.6211	0.9928	0.8336
Gradient Boosting (Tuned)	0.9157	0.7781	0.9176	0.6061	0.9133	0.6123	0.9155	0.6092	0.9777	0.8286
XGBoost (Tuned)	0.9404	0.7831	0.9392	0.6158	0.9419	0.6167	0.9405	0.6163	0.9873	0.8247

Table 7: Tuned Models — Train vs Test Performance Summary

- Random Forest achieved the highest Test ROC-AUC (0.8336) among the tuned models, making it the strongest performer on the primary evaluation metric.
- XGBoost achieved the highest Test Accuracy (0.7831) and Precision (0.6158), indicating better performance in minimizing false positives.
- Random Forest recorded the highest Recall (0.6344), enabling it to identify the largest proportion of actual churners.
- All three tuned models achieved similar overall performance, with differences mainly observed in churn prediction metrics.
- Based on ROC-AUC and Recall, Random Forest emerged as the best-performing tuned model and was selected as the final model.

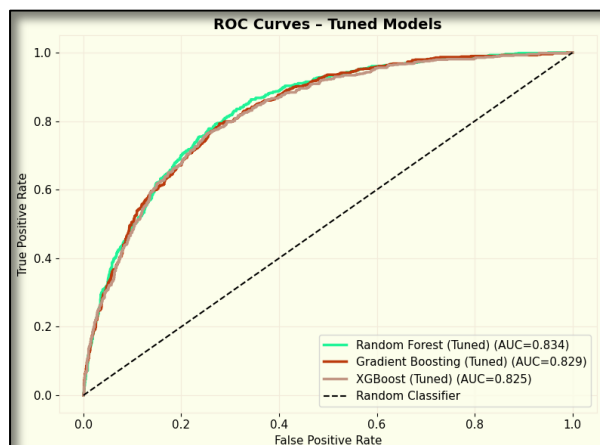


Figure 49: ROC Curves — Tuned Models

Final Model Comparison and Selection

- All three tuned models performed similarly, with Test ROC-AUC values ranging from 0.825 to 0.834.
- Random Forest (Tuned) achieved the highest AUC among the tuned models (0.8336), followed by Gradient Boosting (0.8286) and XGBoost (0.8247).
- However, the untuned Gradient Boosting model achieved the highest overall Test ROC-AUC (0.8358) across all models evaluated.
- Gradient Boosting also maintained a strong balance between Precision, Recall, and F1-score with moderate overfitting.

- Therefore, **Gradient Boosting was selected as the final model** for customer churn prediction due to its superior overall performance and generalization ability.

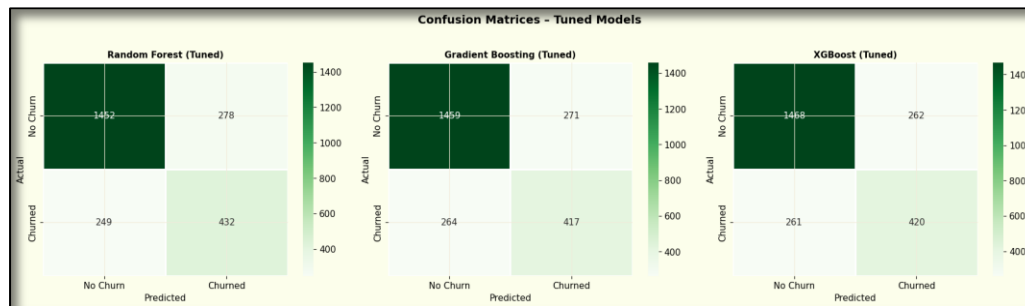


Figure 50: Confusion Matrices — Tuned Models

- Random Forest (Tuned) achieved the highest Recall (0.6344), catching the most churners (432) and missing the fewest (249).
- XGBoost (Tuned) produced the fewest false positives (262) and the highest Precision (0.6158), making it the most efficient model for retention outreach.
- Gradient Boosting (Tuned) delivered balanced performance but trailed the other two models on most test metrics.
- The results highlight a Recall–Precision trade-off: Random Forest identifies more churners, while XGBoost reduces unnecessary retention efforts.
- Overall, Random Forest (Tuned) is preferable when maximising churn detection is the priority, whereas XGBoost (Tuned) is preferable when minimising false positives is more important.

13. Model Comparison & Final Model Selection.

=== FULL MODEL COMPARISON ===					
	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Gradient Boosting	0.7793	0.5971	0.6725	0.6326	0.8358
Random Forest (Tuned)	0.7814	0.6085	0.6344	0.6211	0.8336
Gradient Boosting (Tuned)	0.7781	0.6061	0.6123	0.6092	0.8286
AdaBoost	0.7710	0.5780	0.7019	0.6340	0.8253
XGBoost (Tuned)	0.7831	0.6158	0.6167	0.6163	0.8247
XGBoost	0.7769	0.6032	0.6138	0.6084	0.8228
Logistic Regression	0.7715	0.5827	0.6725	0.6244	0.8225
Random Forest	0.7702	0.5941	0.5888	0.5914	0.8209
Bagging	0.7698	0.5960	0.5742	0.5849	0.8083
Decision Tree	0.7192	0.5027	0.5404	0.5209	0.6650

Figure 51: Full Model Comparison Ranked by ROC-AUC

- Gradient Boosting (untuned) achieved the highest ROC-AUC (0.8358), making it the best overall model for churn prediction.
- Random Forest (Tuned) ranked second with an AUC of 0.8336, while Gradient Boosting (Tuned) ranked third with 0.8286.
- AdaBoost achieved the highest Recall (0.7019), whereas XGBoost (Tuned) achieved the highest Precision (0.6158) and Accuracy (0.7831).
- Logistic Regression remained highly competitive, demonstrating that a simple model can perform comparably to more complex algorithms.
- Boosting-based models consistently outperformed bagging-based models, indicating superior generalization on this dataset.
- Based on its highest ROC-AUC and balanced Precision-Recall performance, **Gradient Boosting (untuned) was selected as the final model.**

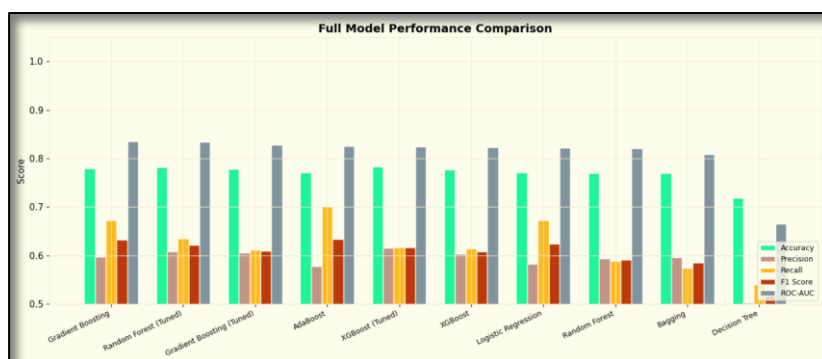


Figure 52: Full Model Performance Comparison

- ROC-AUC remained the strongest metric across all models, supporting its use as the primary evaluation criterion.
- Gradient Boosting achieved the highest ROC-AUC (0.8358), confirming its position as the best overall model.
- AdaBoost recorded the highest Recall (0.7019), making it the most effective at identifying churners.
- XGBoost (Tuned) achieved the highest Precision (0.6158), indicating the lowest false-positive rate among top-performing models.
- Decision Tree performed substantially worse than all other models across every metric, confirming severe overfitting and poor generalisation.
- The top-performing models showed very similar results, highlighting the importance of evaluating multiple metrics rather than Accuracy alone.
- Overall, the comparison confirms Gradient Boosting as the most balanced and reliable model for customer churn prediction.

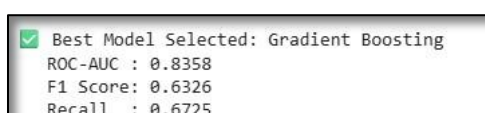


Figure 53: Best Model Selection Summary — Gradient Boosting

- **Gradient Boosting was selected as the final model** as it achieved the highest Test ROC-AUC (**0.8358**) among all models evaluated, indicating the strongest ability to distinguish churners from nonchurners.
- The model also delivered a strong balance between **Recall (0.6725)** and **Precision (0.5971)**, ensuring effective churn detection without excessive false alarms.
- Compared with tuned alternatives, Gradient Boosting maintained the best overall combination of ranking performance, churn detection capability, and generalization on unseen data.
- **Business impact:** the model can be used to identify high-risk customers for proactive retention campaigns, helping reduce customer churn and improve long-term revenue retention.

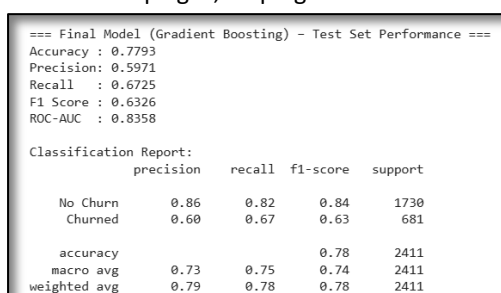


Figure 54: Final Model (Gradient Boosting) — Test Set Performance

- **Final test performance (Gradient Boosting):** ROC-AUC = **0.8358**, Accuracy = **0.7793**, Precision = **0.5971**, Recall = **0.6725**, and F1 Score = **0.6326**.
- The model correctly identified **458 of 681 churners**, achieving a strong balance between churn detection and false positives.
- Performance on the "No Churn" class was also strong (Precision = **0.86**, Recall = **0.82**), ensuring efficient targeting of retention efforts.
- These results confirm that Gradient Boosting provides the best overall balance of predictive accuracy and business value, making it the final model selected for deployment.

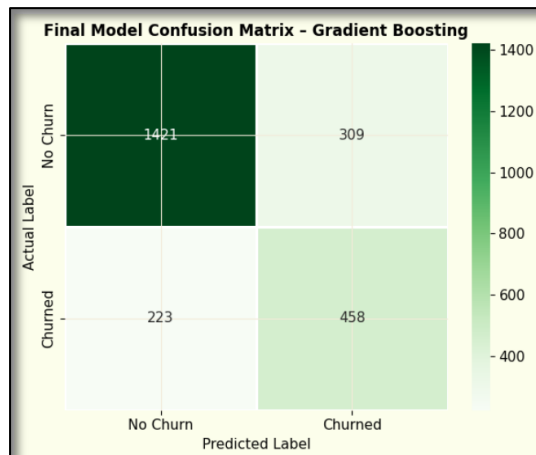


Figure 55: Final Model Confusion Matrix — Gradient Boosting

- **Confusion Matrix:** TN = 1,421, TP = 458, FP = 309, FN = 223.
- The model correctly identifies **458 churners (67.3%)** while missing **223 churners**, providing strong churn detection performance.
- Of the **767 customers flagged as high-risk**, **59.7% are actual churners**, showing efficient targeting compared to random outreach.
- The model correctly excludes **1,421 loyal customers** from retention campaigns, helping reduce unnecessary retention costs.
- Overall, the model achieves a practical balance between identifying churners and controlling false alarms, making it suitable for real-world retention campaigns.

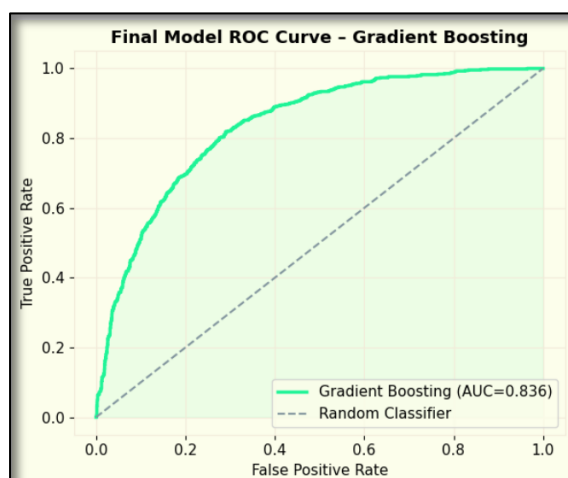


Figure 56: Final Model ROC Curve — Gradient Boosting

- **ROC-AUC = 0.836**, indicating strong ability to distinguish churners from non-churners on unseen data.
- The ROC curve rises steeply at low false positive rates, showing the model can identify most churners while keeping false alarms relatively low.
- The curve remains well above the random-classifier baseline, confirming strong predictive performance across all thresholds.
- The model provides flexibility to adjust the classification threshold based on business priorities, balancing churn detection and retention costs.
- Overall, the ROC curve validates Gradient Boosting as a reliable and deployment-ready model for customer churn prediction.

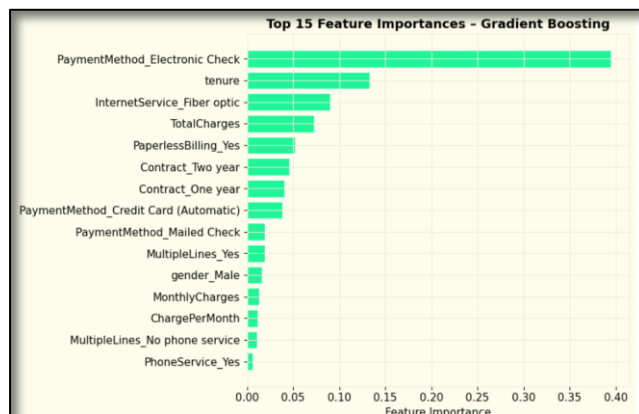


Figure 57: Top 15 Feature Importances — Gradient Boosting

- **PaymentMethod_Electronic Check** is the most important feature, confirming it as the strongest predictor of churn and the highest-priority retention target.
- **tenure**, **InternetService_Fiber optic**, and **TotalCharges** are the next most influential features, reinforcing that customer tenure, service type, and customer value strongly impact churn risk.
- **Contract type** and **automatic payment methods** also play a significant role, with longer contracts and auto-pay associated with lower churn.
- Features such as **gender**, **PhoneService**, **MultipleLines**, and **MonthlyCharges** have relatively low importance and contribute less to churn prediction.
- The feature importance results closely align with the EDA findings, validating the model and highlighting key areas for retention efforts: Electronic Check users, new customers, and Fibre optic subscribers.

14. Key Insights

1. **Month-to-month customers** have the highest churn rate (46%), making them the largest at-risk segment.
2. **Electronic Check users** show the highest churn (49.1%) and are the most important predictor in the final model.
3. **Fiber optic customers** experience high churn (44.9%), indicating potential service quality or value perception issues.
4. Customers without **OnlineSecurity** or **TechSupport** are significantly more likely to churn.
5. **New customers (0–3 months tenure)** face the highest churn risk, highlighting the importance of early engagement.

6. **Senior citizens and customers without partners/dependents** show elevated churn rates.
7. The final **Gradient Boosting model** achieved a ROC-AUC of **0.836**, enabling effective identification of high-risk customers.

Recommendations

1. Encourage **month-to-month customers** to move to longer-term contracts through targeted incentives.
2. Promote **automatic payment methods** among Electronic Check users to improve retention.
3. Investigate and improve the **Fiber optic customer experience** through service quality audits and customer feedback.
4. Bundle **OnlineSecurity and TechSupport** with plans to increase customer stickiness.
5. Implement a **90-day onboarding program** for new customers to reduce early-life churn.
6. Offer targeted retention programs for **senior citizens and vulnerable customer segments**.
7. Deploy the **Gradient Boosting model** for monthly churn scoring and proactive retention campaigns.
8. Monitor model performance regularly and retrain periodically to maintain accuracy.

Appendix

Appendix Index

- A. Library Imports & Setup
- B. Data Preprocessing (Key Steps)
- C. Feature Engineering, Encoding & Split
- D. Model Evaluation Function
- E. Model Training
- F. Hyperparameter Tuning
- G. Final Model Selection
- H. Error Analysis & Diagnostics

Full code execution, intermediate outputs, and complete classification reports for every model are available in the accompanying executed notebook: <LearnerName>_ChurnPrediction_Notebook.html

Appendix A: Library Imports & Setup

Dataset source: AlphaCom customer churn dataset, 12,055 records, provided as part of the [course/program name] capstone project. Not externally/publicly sourced.

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.model_selection import train_test_split, RandomizedSearchCV, cross_val_score

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
                               AdaBoostClassifier, GradientBoostingClassifier)

from xgboost import XGBClassifier

from imblearn.over_sampling import SMOTE

from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                              f1_score, roc_auc_score, roc_curve, confusion_matrix, classification_report,
                              precision_recall_curve)
```

Appendix B: Data Preprocessing (Key Steps)

```
data['Churn'] = data['Churn'].str.strip().str.capitalize()

for col in ['MonthlyCharges', 'TotalCharges']:
    data[col] = pd.to_numeric(data[col].astype(str).str.replace('$', ''), errors='coerce')

data.loc[data['tenure'] < 0, 'tenure'] = np.nan
data = data.drop_duplicates()

data[['tenure', 'MonthlyCharges', 'TotalCharges']] = data[
    ['tenure', 'MonthlyCharges', 'TotalCharges']].fillna(
    data[['tenure', 'MonthlyCharges', 'TotalCharges']].median())

# IQR capping applied to all three numeric columns
for col in ['tenure', 'MonthlyCharges', 'TotalCharges']:
    Q1, Q3 = data[col].quantile([0.25, 0.75])
```

```
data[col] = data[col].clip(Q1 - 1.5 * (Q3 - Q1), Q3 + 1.5 * (Q3 - Q1))
```

Appendix C: Feature Engineering, Encoding & Split

```
data["TenureGroup"] = pd.cut(data["tenure"], bins=[-1, 12, 24, 48, 100],
```

```
labels=['0-12m', '13-24m', '25-48m', '49+m'])
```

```
data["ChargePerMonth"] = data["TotalCharges"] / (data["tenure"] + 1)
```

```
data_enc = pd.get_dummies(data.drop(columns=['Churn']),
```

```
columns=cat_cols, drop_first=True).dropna()
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=42, stratify=y)
```

```
X_res, y_res = SMOTE(random_state=42).fit_resample(X_train, y_train)
```

```
# Scaling — fit on training data only, transform test set (no leakage)
```

```
scaler = StandardScaler()
```

```
X_res_sc = scaler.fit_transform(X_res)
```

```
X_test_sc = scaler.transform(X_test)
```

Appendix D: Model Evaluation Function

```
def evaluate_model(name, model, X_train, y_train, X_test, y_test, overfit_threshold=0.05):
```

```
    """
```

```
    Trains/evaluates a classifier and returns Train vs Test metrics.
```

```
    Flags a metric as 'Overfit' if (Train - Test) > overfit_threshold.
```

```
    """
```

```
    y_train_pred = model.predict(X_train)
```

```
    y_test_pred = model.predict(X_test)
```

```
    y_train_proba = model.predict_proba(X_train)[:, 1]
```

```
    y_test_proba = model.predict_proba(X_test)[:, 1]
```

```
    metrics = {
```

```
        'Accuracy': (accuracy_score(y_train, y_train_pred), accuracy_score(y_test, y_test_pred)),
```

```
        'Precision': (precision_score(y_train, y_train_pred), precision_score(y_test, y_test_pred)),
```

```
        'Recall': (recall_score(y_train, y_train_pred), recall_score(y_test, y_test_pred)),
```

```
        'F1': (f1_score(y_train, y_train_pred), f1_score(y_test, y_test_pred)),
```

```
        'ROC-AUC': (roc_auc_score(y_train, y_train_proba), roc_auc_score(y_test, y_test_proba)),
```

```
    }
```

```
    print(f"\n{'='*55}\n {name}\n{'='*55}")
```

```
    print(f"{'Metric':<12}{Train':>10}{Test':>10}{Overfit?':>12}")
```

```
    results = {}
```

```
    for metric, (train_val, test_val) in metrics.items():
```

```
        gap = train_val - test_val
```

```
        flag = "Overfit" if gap > overfit_threshold else "OK"
```

```
        print(f"{'metric':<12}{train_val:>10.4f}{test_val:>10.4f}{flag:>12}")
```

```

        results[metric] = {'Train': train_val, 'Test': test_val, 'Gap': gap, 'Flag': flag}

    print("\nTest Classification Report:")

    print(classification_report(y_test, y_test_pred, target_names=['No Churn', 'Churned']))

    return results

```

Appendix E: Model Training

Baseline

```
lr = LogisticRegression(max_iter=1000, random_state=42).fit(X_res_sc, y_res)
```

Advanced models (all fitted on X_res_sc, y_res — SMOTE-balanced, scaled training data)

```

models = {

    'Decision Tree': DecisionTreeClassifier(random_state=42),

    'Bagging': BaggingClassifier(n_estimators=100, random_state=42),

    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),

    'AdaBoost': AdaBoostClassifier(n_estimators=100, random_state=42),

    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, random_state=42),

    'XGBoost': XGBClassifier(eval_metric='logloss', random_state=42),

}

```

```
for name, model in models.items():
```

```
    model.fit(X_res_sc, y_res)
```

```
    evaluate_model(name, model, X_res_sc, y_res, X_test_sc, y_test)
```

Appendix F: Hyperparameter Tuning

Search method: RandomizedSearchCV, cv=5, scoring='roc_auc', n_iter=20 per model (100 fits each, 300 total).

```

rf_params = {

    'n_estimators': [100, 150, 200, 300],

    'max_depth': [5, 8, 10, 15, None],

    'min_samples_split': [2, 5, 10],

    'min_samples_leaf': [1, 2, 4],

    'max_features': ['sqrt', 'log2'],

}

```

```

gb_params = {

    'n_estimators': [100, 150, 200, 300],

    'learning_rate': [0.01, 0.05, 0.1, 0.2],

    'max_depth': [3, 4, 5, 6],

    'subsample': [0.6, 0.7, 0.8, 1.0],

    'min_samples_split': [2, 5, 10],

}

```

```

xgb_params = {

    'n_estimators': [100, 150, 200, 300],

    'learning_rate': [0.01, 0.05, 0.1, 0.2],

    'max_depth': [3, 4, 5, 6],

}

```

```

'subsample':      [0.6, 0.7, 0.8, 1.0],
'colsample_bytree': [0.6, 0.8, 1.0],
'gamma':          [0, 0.1, 0.3, 0.5],
'reg_alpha':      [0, 0.1, 1],
}

search_configs = {
    'Random Forest': (RandomForestClassifier(random_state=42), rf_params),
    'Gradient Boosting': (GradientBoostingClassifier(random_state=42), gb_params),
    'XGBoost': (XGBClassifier(eval_metric='logloss', random_state=42), xgb_params),
}

tuned_models = {}

for name, (estimator, params) in search_configs.items():
    search = RandomizedSearchCV(estimator, param_distributions=params,
                                n_iter=20, cv=5, scoring='roc_auc', random_state=42, n_jobs=-1, verbose=1)
    search.fit(X_res_sc, y_res)
    tuned_models[name] = search.best_estimator_
    print(f"Best {name} params: {search.best_params_} | CV AUC: {search.best_score_:.4f}")

```

Appendix G: Final Model Selection

```

all_df = pd.DataFrame(all_results).T.sort_values('ROC-AUC', ascending=False)
best_model_name = all_df['ROC-AUC'].idxmax()
best_model = tuned_models.get(best_model_name, models.get(best_model_name, lr))

```

Appendix H: Error Analysis & Diagnostics

H.1 False Negative Profile (Missed Churners)

To understand where the final Gradient Boosting model fails, the 223 false negatives on the test set are profiled against key features such as tenure, contract type, and payment method, to check whether errors cluster in a specific customer segment rather than occurring randomly.

```

fn_mask = (y_test == 1) & (y_test_pred == 0)
fn_profile = X_test[fn_mask].describe()
print(fn_profile[['tenure', 'MonthlyCharges', 'Contract_Two year',
                  'PaymentMethod_Electronic Check']].mean())

```

Output (mean feature values among the 223 missed churners) is available in the executed notebook, Appendix H section.

H.2 Precision–Recall Tradeoff by Threshold

The report recommends lowering the classification threshold to favour recall. At the default 0.50 threshold, the final model achieves Precision = 0.597, Recall = 0.673, F1 = 0.633, catching 458 of 681 churners (Section 14). The code below recomputes this tradeoff at lower thresholds to quantify how many additional churners would be caught, and at what precision cost, before any threshold change is operationalised.

```

y_proba = best_model.predict_proba(X_test_sc)[:, 1]
precisions, recalls, thresholds = precision_recall_curve(y_test, y_proba)
for t in [0.3, 0.4, 0.5, 0.6]:
    preds_t = (y_proba >= t).astype(int)
    print(f"Threshold {t}: Precision={precision_score(y_test, preds_t):.3f}, "
          f"Recall={recall_score(y_test, preds_t):.3f}, "
          f"F1={f1_score(y_test, preds_t):.3f}")

```

H.3 Cross-Validation Stability of the Final Model

python

```
cv_scores = cross_val_score(best_model, X_res_sc, y_res, cv=5, scoring='roc_auc')
print(f"CV ROC-AUC: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
```

Mean and standard deviation across the 5 folds are available in the executed notebook, Appendix H section, and confirm whether the reported 0.8358 test ROC-AUC reflects a stable result rather than a single favourable split.

H.4 Feature Importance Consistency Across Models

Table H.4 — Top 5 Features by Model (from charts in Sections 10 and 14)

Electronic Check, tenure, Fibre Optic, and TotalCharges appear among the top predictors in both a linear model (Logistic Regression) and a non-linear ensemble (Gradient Boosting), despite the two models using entirely different mechanisms to rank features. This consistency across architectures supports treating these as genuine churn drivers rather than artifacts of one model's internal weighting.

```
python
top_features = {}
for name, m in {'Random Forest': models['Random Forest'],
               'XGBoost': models['XGBoost']}.items():
    top5 = pd.Series(m.feature_importances_, index=X_res.columns).sort_values(ascending=False).head(5)
    top_features[name] = list(top5.index)

pd.DataFrame(top_features)
```